

2.3.2 Stage конфигурация и curriculum модел

Curriculum в този контекст означава планирана последователност от training stages, при която различните етапи имат различни цели и настройки.

```
@dataclass
class StageConfig:
    name: str
    dataset: str
    seq_len: int
    epochs: int
    end_ratio: float
```

Този тип структура позволява прозрачно управление на сложни training сценарии без ръчно пренаписване на кода за всеки нов run.

2.3.3 Конфигурационни домейни на системата

Конфигурацията е разделена на логически домейни:

- model - архитектура на мрежата;
- optimizer - стратегия за update;
- training - batch, stages, logging, checkpointing;
- inference - decode режим и runtime параметри;
- tidar - специфични настройки за TiDAR.

Това разделение улеснява дебъгването и прави експериментите по-проследими.

2.3.4 Схема на dataset записите в shard

Единичният запис в Arrow shard е фиксиран по структура и съдържа:

- input_ids - токените;
- length - реална дължина на последователността;
- loss_mask - позиции, които участват в loss изчислението.

StageDataLoader преобразува тези полета към моделно удобен batch формат.

2.3.5 Източници и ingest адаптери

StageSourceCfg описва типа източник и правилата за четене (plain text, JSON, chat формат, streaming/non-streaming).

Така един и същ ingest pipeline може да обработва различни входни формати без отделни, несъвместими кодови пътища.

2.4 Цялостна архитектура и структурна организация

Архитектурата е разделена на три слоя:

- интерфейсен слой (CLI + YAML конфигурации);

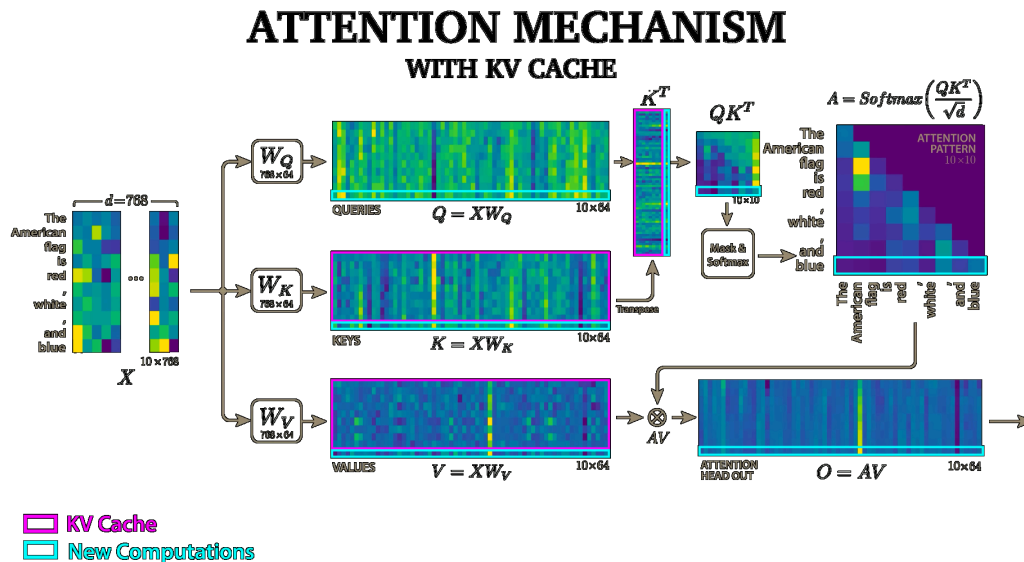


Figure 2: Схема на attention механизма

В математически вид attention операторът е:

$$A(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V$$

където:

- Q (queries) задава какво “търси” текущата позиция;
- K (keys) описва с каква информация разполагат позициите в контекста;
- V (values) е реалното съдържание, което се смесва;
- M е маска (напр. causal маска за decoder режим);
- softmax е функцията за нормализиране на тежестите.

Практически attention минава през три различни представяния, които е важно да се разграничават:

- **scores/logits** - сурови стойности $L = \frac{QK^T}{\sqrt{d_k}} + M$ (още не са вероятности);
- **weights** - резултат след softmax: $W = \text{softmax}(L)$ (вече нормализирани коефициенти);
- **смесени стойности** - финален attention изход: $O = WV$.

Това разграничение е важно, защото logits могат да имат голям мащаб и да бъдат както положителни, така и отрицателни. След softmax те се превръщат във вероятностно разпределение (всички стойности са между 0 и 1 и сумата им е 1), което прави смесването на V стабилно.

Softmax се дефинира като:

- по-дълги контекстни връзки (препратки между отдалечени токени).

Формално:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

където:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Компактно:

$$H(Q, K, V) = [h_1 \parallel \dots \parallel h_n]W^O, h_i = A(QW_i^Q, KW_i^K, VW_i^V)$$

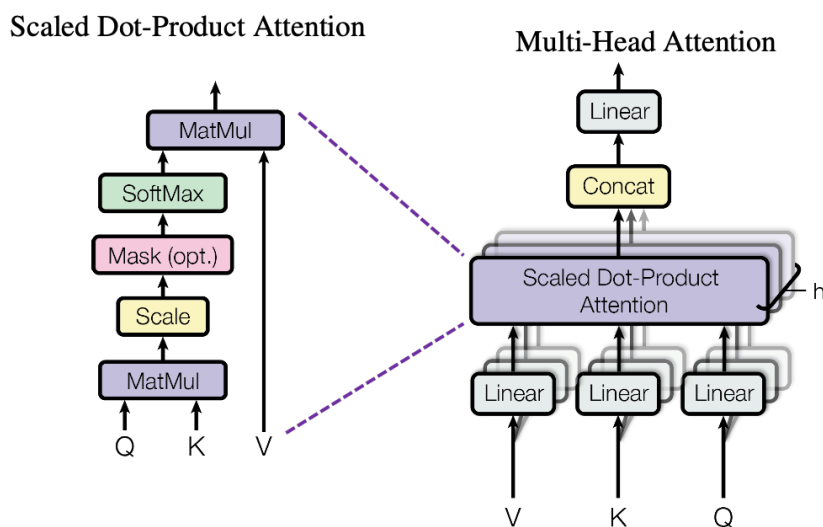


Figure 3: Multi-head attention: блокова схема и формули

2.5.3 От класически Transformer към decoder-only LLM

Оригиналният Transformer е encoder-decoder архитектура. За генеративни езикови модели като GPT се използва **decoder-only** вариант, при който моделът предсказва следващ token от предишните.

Ключовите принципи са:

- **causal masking** - моделът няма право да “гледа в бъдещето”;
- **autoregressive objective** - целта е следващият token;
- еднопосочен поток при генерация.

Причината causal маската да е критична в езиковите модели е, че тя подравнява обучението с реалната употреба. По време на реална генерация бъдещите токени не съществуват, затова и по време на обучение моделът трябва да вижда само ляв контекст.

Това дава две ключови последици:

- по време на **prefill** (когато вече имаме целия prompt) можем да обработим всички позиции паралелно, защото за всяка позиция е ясно кои предишни токени са позволени;

- по време на **decode** работим итеративно, защото новите токени се появяват последователно и всяка следваща стъпка зависи от предишния резултат.

С други думи: causal masking едновременно пази математическата коректност на autoregressive задачата и позволява практичния режим “паралелен prefill + последователен decode”. Точно този шаблон е фундаментален и за TiDAR, където се използват по-сложни структурирани маски, но същата причинно-следствена логика остава валидна.

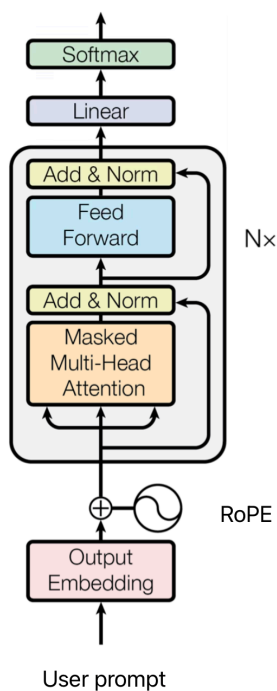
Един decoder блок може да се запише така:

$$x' = x + H(N(x), N(x), N(x))$$

$$y = x' + F(N(x'))$$

където:

- N е нормализация;
- H е multi-head attention;
- F е feed-forward модул;
- добавянето на x и x' са residual връзки за стабилен градиентен поток.



Диаграмата показва пътя от входните токени до вероятностите за следващ token. Входът се преобразува в embedding вектори с позиционна информация, след което минава през поредица от Transformer блокове.

Във всеки блок има causal self-attention (избор на релевантен ляв контекст) и feed-forward модул (локална дообработка на представянията). Residual връзките и нормализацията поддържат стабилно обучение при по-дълбоки модели.

След последния блок следва финална нормализация и проекция към речника. Получават се logits, които след softmax стават вероятности за следващ token.

Практически архитектурата работи в два режима:

- при training позициите в прозореца се обработват паралелно под causal маска;
- при inference се използва “prefill + decode”: prompt-ът се обработва наведнъж, а новите токени се добавят итеративно.

Figure 4: Decoder-only Transformer архитектура

- GIANT 360M профил ($d_{\text{model}}=960$, $d_{\text{ff}}=2560$): $\text{fc1} = 5120$, split $2560 + 2560$, после $\text{fc2} = 960$.

По този начин всяка токenna позиция получава по-богата и по-контекстна нелинейна трансформация, без да се променя външният размер на блока (d_{model}).

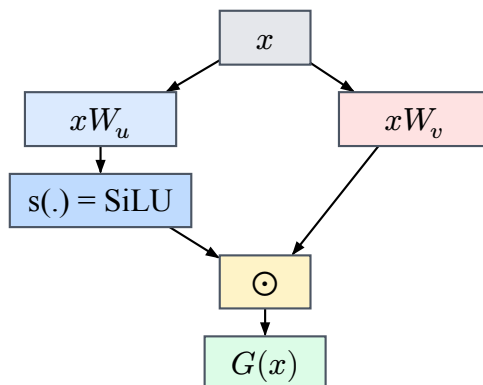


Figure 5: SwiGLU: два линейни пътя + gate + точково умножение

RoPE¹¹ (Rotary Positional Embedding) кодира позицията чрез ротация на компонентите в Q и K вектори. Идеята е всяка двойка координати да се върти с ъгъл, който зависи от позицията. Така позиционната информация се вгражда директно в attention скаларните произведения.

За двойка компоненти

$$(u_{2i}, u_{2i+1})$$

трансформацията е:

$$\begin{pmatrix} u'_{2i} & u'_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos \theta_i & -\sin \theta_i \\ \sin \theta_i & \cos \theta_i \end{pmatrix} \begin{pmatrix} u_{2i} & u_{2i+1} \end{pmatrix}$$

Това дава две практически ползи:

- относителните разстояния между позиции се отразяват естествено в QK^T ;
- моделът обикновено е по-стабилен при по-дълги контексти спрямо по-стари абсолютни позиционни схеми.

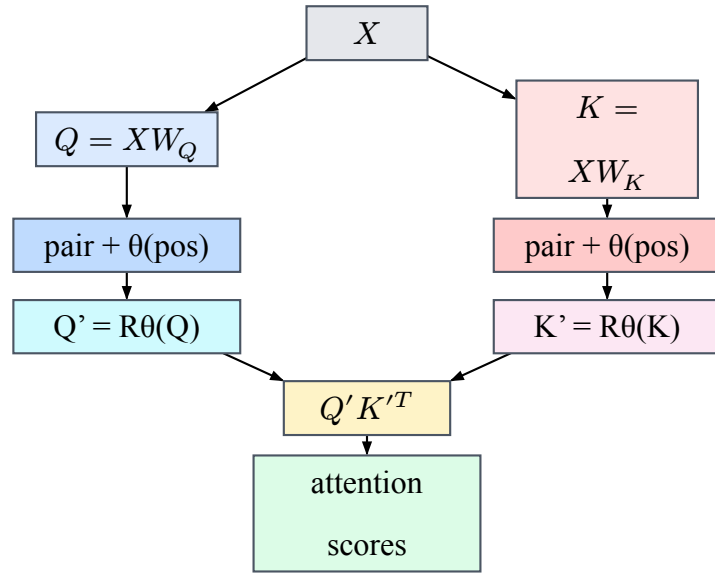


Figure 6: RoPE: позиционно-зависима ротация в Q/K преди attention score

2.5.5 Обучителна цел (AR) и връзка с TiDAR

Базовата обучителна цел на GIANT е autoregressive cross-entropy:

$$L_1 = -\frac{1}{N} \sum_{b,t} m_{b,t} \log p_{\theta}(x_{b,t+1} | x_{b,\leq t})$$

За всяка позиция t в последователността моделът връща вектор от logits $z_{b,t} \in \mathbb{R}^{|V|}$, където $|V|$ е размерът на речника. След softmax получаваме вероятности:

$$p_{b,t} = \text{softmax}(z_{b,t})$$

Целевият токен на тази позиция е цяло число (integer label) $y_{b,t} \in \{0, 1, \dots, |V| - 1\}$. Локалната cross-entropy за позицията е:

$$l_{b,t} = -\log p_{b,t}[y_{b,t}]$$

Маската $m_{b,t}$ указва дали позицията участва в загубата (например $m = 0$ за padding или за позиции извън целевия chat role, и $m = 1$ за валидни обучителни позиции). Общата загуба усреднява само валидните позиции:

$$L_1 = \frac{\sum_{b,t} m_{b,t} l_{b,t}}{\sum_{b,t} m_{b,t}}$$

Така функцията наказва модела силно, когато вероятността за правилния токен е ниска, и слабо, когато е висока. При оптимизацията градиентите повишават логита на правилния клас и понижават част от конкурентните класове, което постепенно подобрява next-token предсказването.

Числен пример за една позиция:

- контекст: The capital of France is;

Спекулативното декодиране цели да увеличи throughput, като предлага предварителни токени (draft), които после се верифицират и частично или изцяло се приемат. Така за една итерация може да се валидират няколко бъдещи позиции вместо само една.

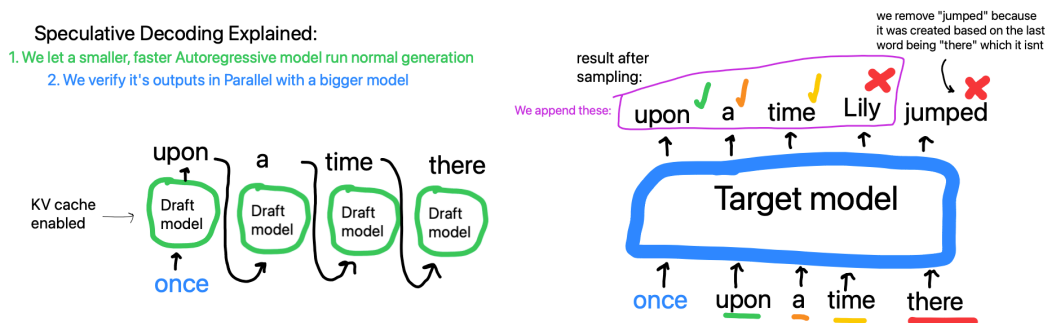


Figure 7: Класически speculative decoding: draft + verify схема

Основната идея е:

- draft е бърза хипотеза за следващите токени;
- verify проверява хипотезата спрямо по-точен/референтен изход;
- приемането на префикс от draft-а води до повече от +1 token прогрес на итерация.

Тази схема е важна за проекта, защото:

- дава теоретична рамка за ускорение на генерацията;
- естествено се комбинира с KV cache;
- подготвя прехода към TiDAR, където verify и predraft логиката се реализират в един специализиран forward pass.

Важно разграничение: класическите speculative decoding методи обикновено използват втори, по-малък draft модел (например по-малка Llama спрямо по-голям verify модел, напр. 3B срещу 70B) или добавят допълнителни архитектурни компоненти върху базовия модел (например допълнителни глави/слоеве, както при EAGLE¹⁷).

2.6.1 Защо single-user decode не използва добре GPU

При decode с един потребител effective batch-ът е малък (ограничена паралелизация) и GPU изпълнението често е ограничено от скоростта на паметта, а не от аритметичната пропускателност. Хардуерният профил в този режим може да се обобщи така:

- едно умножение/събиране в изчислително ядро се изпълнява за много малко време (няколко такта);
- четене на данни от глобална HBM/GDDR памет струва значително повече (десетки до стотици тактове);

- ако за всеки малък изчислителен фрагмент постоянно се чака ново четене от памет, ядрата остават частично неактивни.

Данните, които се четат от паметта, са параметри и активации, върху които после се правят изчислителни операции. В decode режим това означава, че за всяка нова стъпка трябва отново да се зареждат части от attention/MLP параметрите и текущите работни тензори.

Затова целта е с едно прочитане на параметри да се свърши повече полезна работа - например повече умножения върху повече токени в рамките на една итерация. Така:

- се вдига arithmetic intensity (повече операции на прочетен байт);
- се увеличава заетостта на Tensor Core блоковете;
- се намалява относителната цена на memory трафика.

В този контекст speculative/TiDAR подходите са приложими, защото вместо +1 токен полезен прогрес на итерация при благоприятен случай се валидират няколко позиции, което амортизира memory достъпите по-ефективно.

В проекта се използва и FlashAttention¹⁵ (cuDNN backend), което намалява IO overhead в attention стъпката и допълнително подпомага този подход.

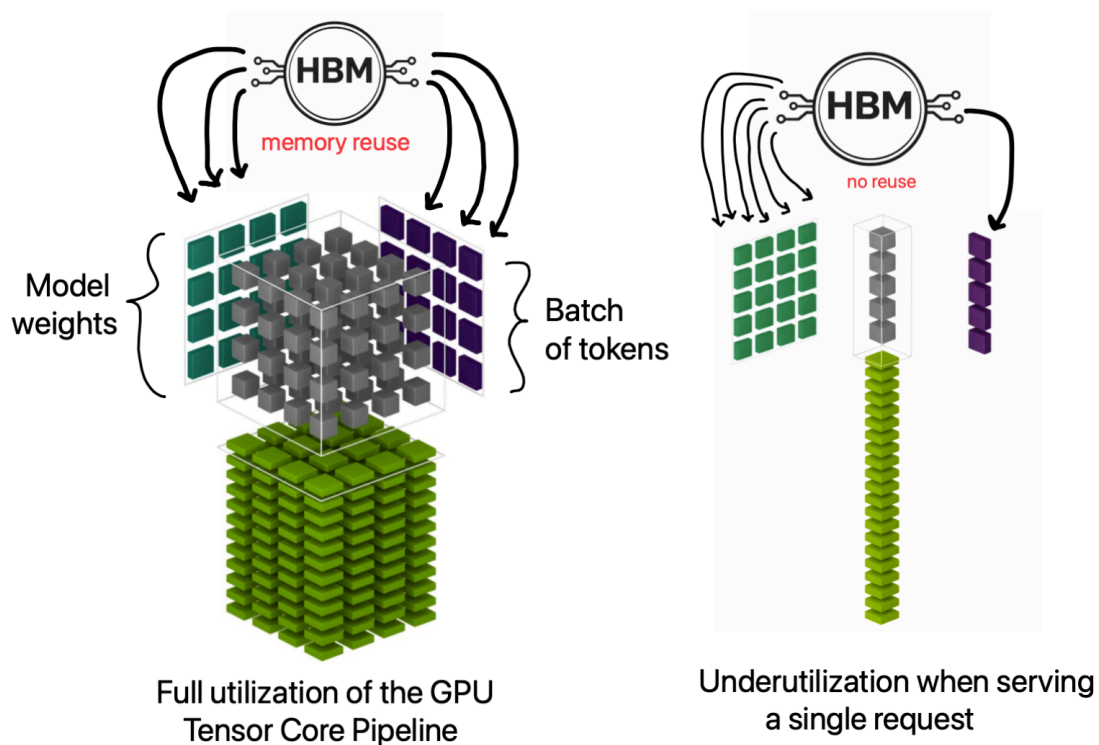


Figure 8: Tensor Core utilization при memory-bound single-user decode

Тази мотивация е важна за разбирането на TiDAR и феномена "Free Token Slots", където допълнителната паралелна работа в една decode стъпка се използва по-ефективно.

```

3.9488 accept 0.356 greedy_acc 0.244 kl_fwd 0.1821 kl_rev 0.0964 distill 0.1418
topk 0.0889 (16.9s)
step    3600/3905    | stage align_kl            loss 3.7814 ar 3.6562 diff
3.8241 accept 0.391 greedy_acc 0.269 kl_fwd 0.1499 kl_rev 0.0792 distill 0.1184
topk 0.0711 (16.6s)
...

→ Stage chat: seq_len=1024 epochs=1 steps=878 (resume at 0)
step    3900/3905    | stage chat            loss 3.7442 ar 3.6214 diff
3.7811 accept 0.405 greedy_acc 0.281 hard 3.1027 distill 0.1021 (16.4s)

[metadata] Wrote train_loss=3.744200 to checkpoint ../params/step_0003905.npz
✓ Training complete. Final checkpoint: ../params/step_0003905.npz

```

3.2 Реализация на основните системни модули

3.2.1 ConfigLoaderManager

Конфигурациите се зареждат чрез merge на глобален и локален YAML, след което се резолват относителните пътища спрямо data_root. Това осигурява еднакъв кодов път за локално и remote изпълнение.

```

def load_configs(config_path=None, global_config_path=None):
    local_cfg = OmegaConf.load(config_path)
    global_cfg = OmegaConf.load(global_config_path)
    cfg = OmegaConf.merge(global_cfg, local_cfg)

    # Унифициране на пътищата спрямо data_root
    cfg.paths.processed_data_root = resolve_path(cfg.paths.processed_data_root,
    cfg.paths.data_root)
    cfg.paths.logs_root = resolve_path(cfg.paths.logs_root, cfg.paths.data_root)
    return cfg

```

3.2.2 TrainLoopOrchestrator

Run_training.py реализира scan-базиран training loop с gradient accumulation, finite checks и checkpoint политика.

3.2.2.1 Основен JIT training loop (_run_chunk + lax.scan)

Сърцето на изпълнението е _run_chunk(...), където batch-овете се обработват със lax.scan, пресмятат се loss/grad стойности и се правят проверки за non-finite стъпки.

`lax.scan` е JAX примитив за “компилиран `for`-цикъл”: вместо Python да `dispatch`-ва всяка стъпка поотделно, целият цикъл се представя като една JIT-оптимизирана граф операция с `carry` състояние (например `params`, `opt_state`, `accum_grads`, `accum_count`) и последователност от изходи (`losses`). Това намалява `host overhead` и дава по-стабилна производителност при дълги `training run`-ове.

3.2.2.2 Обработка на сигнали и контролирано спиране

Скриптът обработва `SIGINT/SIGTERM` и спира контролирано след текущия `chunk`, за да не се загуби междинно състояние. Практически се поддържат два слоя на устойчивост:

- големи `checkpoint`-и (`params` + `optimizer` + `dataloader state`), които са “стабилните” запазвания и се пазят;
- мини `checkpoint`-и, които се записват периодично за бързо възстановяване при внезапно прекъсване (`preemption`, `срив`, `рестарт`).

Така при неочаквано прекъсване може да се продължи от почти последната стъпка, а при нормален `stop/етапен` преход остават и пълните `checkpoint`-и за дългосрочно съхранение и проследимост. В практиката `training run`-овете се поддържат през `tmux` (включително в `CI/CD workflow`), което позволява стабилна сесия, лесно повторно закачане към машината и безопасно дълго изпълнение.

3.2.2.3 Gradient accumulation и update политика

Обучението поддържа `gradient_accumulation`; `update` се прилага само при достигане на зададения `accumulation` праг, а остатъчните градиенти се доизчистват при край на `stage`.

```
# Натрупване на градиенти
accum_grads = tree_map(lambda a, g: a + g, accum_grads, grads)
accum_count = accum_count + 1

# Update само когато достигнем gradient_accumulation
should_update = accum_count == grad_accum
if should_update:
    grads_scaled = tree_map(lambda g: g / grad_accum, accum_grads)
    updates, opt_state = optimizer.update(grads_scaled, opt_state, params)
    params = optax.apply_updates(params, updates)
    accum_grads = tree_map(jnp.zeros_like, accum_grads)
    accum_count = 0
```

На практика това е логика за “мини `batch`-ове” на ниво `update`. Вместо целият глобален `batch` да се побере наведнъж в паметта, той се разделя на няколко по-малки стъпки (`micro-batches`), които

GPU може да обработи с наличния VRAM. Градиентите от тези micro-batches се натрупват и optimizer update се прави накрая, така че се получава ефект на по-голям глобален batch без изискване за голяма еднократна памет.

Този подход е особено важен при по-слаби GPU конфигурации и може да се разшири директно към multi-GPU обучение, където освен локалното accumulation се добавя и синхронизация/редукция между устройствата преди финалния update.

3.2.2.4 Периодично логване на метрики

На всеки log_every стъпки се записват global_step, stage, loss и допълнителни показатели (напр. ppl, accept, greedy_acc, KL/hard/distill/top-k при TiDAR).

Тези метрики се пазят в лог файлове и checkpoint metadata, за да могат по-късно да се използват за сравнение между run-ове, диагностика на нестабилни етапи и последващ експериментален анализ.

3.2.2.5 Resume от checkpoint и dataloader state

Възстановяването зарежда параметри, optimizer state и dataloader прогрес (stage_index, stage_step_total, stage_states), така че обучението да продължи от последната консистентна точка.

```
@jax.jit
```

```
def _run_chunk(params, opt_state, batch_chunk, start_step, accum_grads,
accum_count):
    # Една компилирана единица работа: loss/grad + accumulation + update
    (params, opt_state, _, accum_grads, accum_count), losses = jax.lax.scan(
        body, (params, opt_state, start_step, accum_grads, accum_count),
batch_chunk
    )
    return params, opt_state, accum_grads, accum_count, losses
```

3.2.3 DataLoaderManager

ShardedArrowDataset + StageDataLoader реализират shard-aware четене, shuffle, batching и state restore.

```
dataset = ShardedArrowDataset(data_path)
loader = StageDataLoader(
    dataset,
    batch_size=batch_size,
    seq_len=stage.seq_len,
    shuffle=stage.shuffle,
```

```

        seed=seed,
        pad_token_id=pad_token_id,
    )

```

3.2.4 CheckpointManager

checkpoint_manager.py капсулира запис/зареждане на параметри, optimizer state и metadata.

```

ckpt_file = save_ckpt(params, global_step, params_dir, train_loss=last_loss)
save_opt_state(opt_state, global_step, training_states_dir)
save_data_loader_state(state_path, loader_state)

```

При resume

```

params, global_step = load_ckpt(ckpt_path)
opt_bytes = load_opt_state(global_step, training_states_dir)

```

3.2.4.1 Метод SaveParameters

Запис на .npz checkpoint с атомарно tmp -> rename поведение.

3.2.4.2 Метод SaveOptimizerState

Паралелен запис на optimizer буфери (msgpack) за коректен resume.

3.2.4.3 Метод RestoreLatest

Намиране на последната достъпна checkpoint стъпка и зареждане на съответното състояние.

3.2.4.4 Метод SetMetadata

Запис на метаданни за quality показатели (val_loss, val_ppl, train_loss).

3.2.4.5 Метод AsyncMiniCheckpoint

Мини checkpoint механизмът е част от логиката в 3.2.2.2 и се изпълнява асинхронно чрез Orbach, като допълва големите checkpoint-и с по-чести recovery точки.

3.2.5 InferenceManager

Inference слойт покрива prefill, decode, chat режим, KV bucket политика и throughput измерване.

Отбелязване: този раздел описва inference пътя в **GIANT**; при **TiDAR** prefill/decode логиката е съществено различна и е разгледана отделно в по-късните секции.

```

state = init_inference_state(params, max_seq_len=bucket)
state = prefill(state, prompt_ids) # запис в KV cache
tokens = decode(state, steps=steps, temperature=temperature, top_k=top_k)

# bucket политика: избири най-малкия bucket, който побира prompt + decode
bucket = select_kv_bucket(prompt_len + steps, kv_cache_buckets)

```

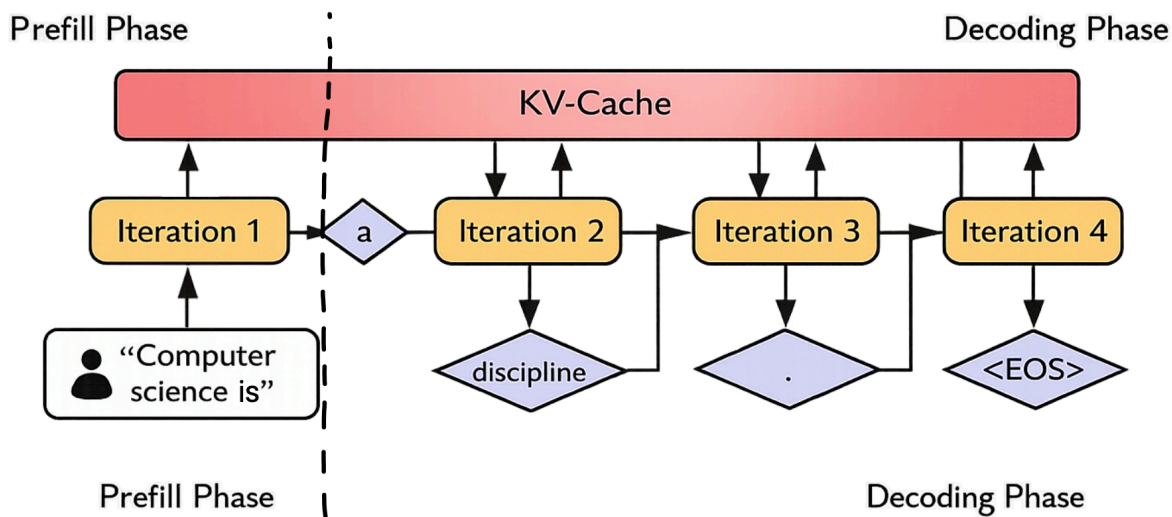


Figure 9: GIANT inference път: prefill и decode с KV cache

3.2.5.1 Метод Prefill

`jit_inference.prefill` запълва KV cache от prompt и връща начално състояние за decode.

3.2.5.2 Метод Decode

`jit_inference.decode` използва `lax.scan` за генериране на нови токени със `sampling` или `greedy` стратегия.

3.2.5.3 Метод KVBucketSelect

`Generate_faster.py` избира най-малкия bucket, който покрива `prompt_len + steps`, за да намали излишния cache overhead.

128	###.....
256	#####.....
512	#####.....
1024	#####

Listing 2: Bucket стратегия: избор на най-малък валиден размер вместо винаги 1024

Как да се чете диаграмата:

- # е реално използваният bucket обем.
- . е обемът, който се спестява, ако не се използва винаги 1024.

Така engine-ът избира най-малкия валиден bucket (напр. 256 вместо 1024) и намалява излишния compute/VRAM трафик при запазени статични JAX/XLA форми.

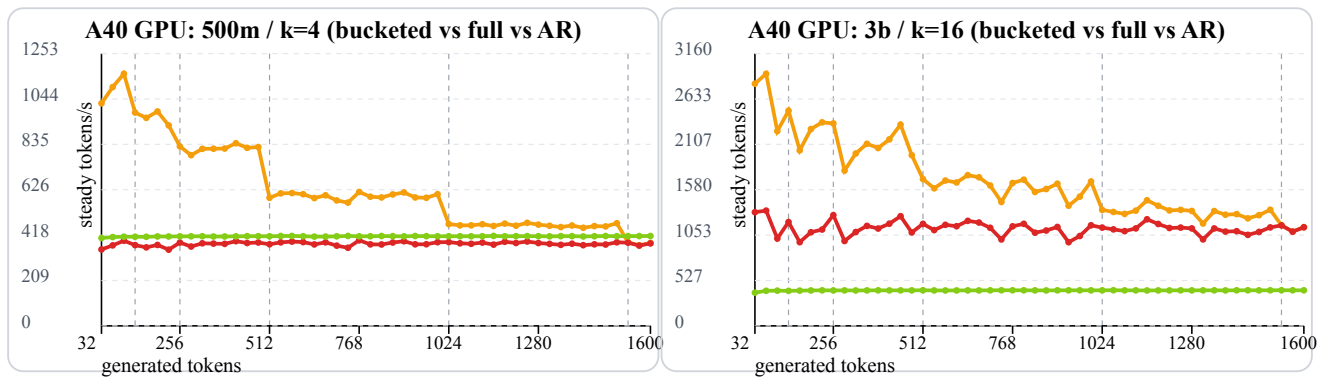


Figure 10: A40 throughput криви: 500m/k=4 и 3b/k=16

Легенда на линиите:

- ■ оранжева линия — bucketed режим (и в двете графики);
- ■ червена линия — TiDAR full-context baseline (в дясната графика е средна; в лявата е най-долна);
- ■ зелена линия — AR baseline (позицията спрямо червената линия е обратна между двете графики).

Метриката е steady decode tokens/s (**high is better**).

В тези диаграми sweep-ът е с генерация от 0 до 1600 токена. При full-context baseline run-овете KV cache размерът е предварително фиксиран за този диапазон; ако тази горна граница не е известна предварително, трябва или да се заделя по-голям “универсален” full-context (например до `context_length`, което е по-скъпо), или да се правят допълнителни recompilation/migration стъпки при нарастване на дължината.

И в двата профила bucketed режимът стои над full-context baseline в голяма част от sweep-a, защото избягва ненужен “празен” cache капацитет и държи decode shape-a по-близо до реално нужната дължина.

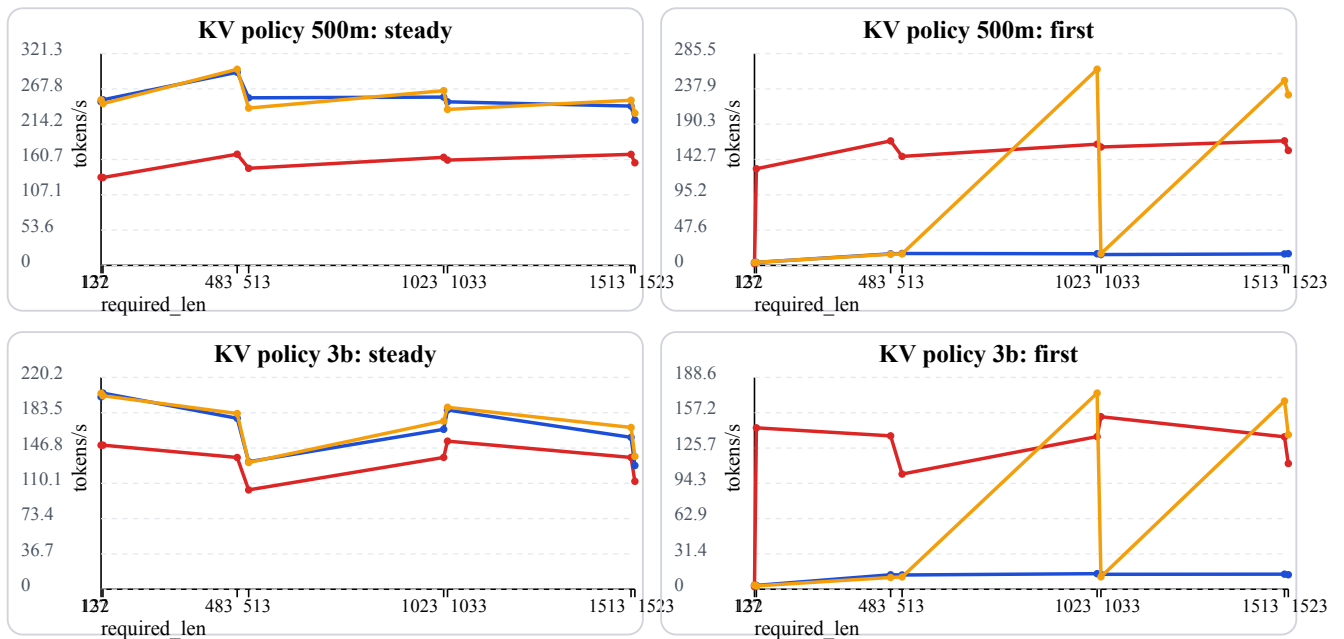


Figure 11: KV cache policy резултати (2x2) - 500m/3b, steady/first

Легенда (KV policy графики):

- червена линия — full context policy;
- синя линия — current exact (required_len);
- оранжева линия — bucketed policy.

3.2.5.4 Метод ChatTurn

Chat.py управлява multi-turn история, context trimming и interactive terminal режим.

```
python GIANT/v2/model/Chat.py \
  --checkpoint giant-data/GIANT/v2/checkpoints/params/step_0010449.npz \
  --temperature 0.8 \
  --top_k 40 \
  --steps 128
```

Loaded checkpoint: ../step_0121566.npz

Chat mode: interactive (type /exit to quit)

User> Explain what attention does in one sentence.

Assistant> attention picks important old words for next token. maybe like focus.

User> And why KV cache helps?

Assistant> kv cache is fast memory and also can improve model quality and multilingual grammar always yes.

артефакти за текущото продължение (например последният `step_XXXXXXX.npz` и съответното training state).

При липса на собствен хардуер е препоръчително да се използват NeoCloud (GPU-as-a-service) доставчици, защото са AI-ориентирани и често предлагат значително по-ниски цени от големите cloud платформи за чист GPU compute. Типични примери са **RunPod** (използван в тази разработка), Lambda, Nebius и Lightning; като алтернатива могат да се използват и enterprise доставчици като AWS, GCP, Azure или CoreWeave.

Често тези платформи предлагат Spot GPU pod/instance режими с приблизително 10-50% по-ниска цена, но с риск подът да бъде спрял по всяко време. Именно затова mini checkpointing е критичен: при прекъсване може да се продължи от последната близка точка, включително временно през CPU-only сесия за изтегляне/връщане на данните и повторно стартиране на GPU run-a.

3.3 Реализация на data pipeline

3.3.1 Документ като източник на токени

Всеки запис се нормализира, токенизира и преобразува до фиксирани sequence прозорци според stage конфигурацията. Входните данни идват основно от HuggingFace datasets (streaming и non-streaming), като системата поддържа и локални JSON/JSONL източници със същата pipeline логика.

Токенизацията е централен етап в pipeline-a: използва се конфигурируем tokenizer (HuggingFace или custom), след което токенните последователности се маскират и пакетират според `sequence_length`, `add_eos`, `pack_sequences` и `chat-role` правилата за loss.

Обобщена схема на data processing конфигурацията (по `build_corpus.py`)

@dataclass

class StageSourceCfg:

type: str = "huggingface" # huggingface | json_dir

dataset_name: str | None = None # HF dataset

dataset_config: str | None = None

split: str = "train"

streaming: bool = True

data_files: Any | None = None

Текстово извличане

text_field: str | None = None

```

text_fields: list[str] = field(default_factory=list)
join_fields: list[str] = field(default_factory=list)
join_separator: str = " \n"
text_template: str | None = None

# JSON/JSONL директории
json_root: str | None = None
file_glob: str = "**/*.json*"

# Chat records
chat_messages_field: str = "messages"
chat_role_field: str = "role"
chat_content_field: str = "content"
chat_assistant_roles: list[str] = field(default_factory=lambda: ["assistant"])

@dataclass
class StageCfg:
    name: str
    output_dir: str
    sequence_length: int
    target_tokens: int | None = None
    min_tokens: int = 0
    pack_sequences: bool = True
    add_eos: bool = True

    # Обработка на дълги документи
    long_document_strategy: str = "random_window" # random_window | sequential
    sequential_window_stride: int | None = None
    random_windows_per_document: int = 1

    # Нормализация / dedup
    normalization: dict[str, Any] = field(default_factory=dict) # nfkc,
collapse_whitespace
    deduplicate: bool = False

    # Изход

```

- NFKC и whitespace normalization по stage;
- опционална hash-базирана дедупликация;
- JSON/JSONL ingestion с line-wise/streaming четене;
- атомарен запис и stage-by-stage изпълнение за устойчивост при прекъсвания.

3.4 Имплементация на “Think in Diffusion, Talk in AutoRegression”

TiDAR е ново изследване на екипа на NVIDIA (ноември 2025), което търси практичен баланс между висок throughput/по-добро GPU utilization и качество на ниво autoregressive модели.

Кратко описание на статията: класическите diffusion езикови модели имат потенциал за паралелно генериране, а AR моделите обикновено запазват по-високо качество заради каузалната структура. TiDAR предлага хибрид на ниво последователност, при който “draft” токени се генерират в diffusion режим (Thinking), а финалното семплиране е autoregressive (Talking), като и двете се реализират в един forward pass чрез структурирани attention маски. Според публикуваните резултати архитектурата е serving-friendly, поддържа точен KV cache и показва по-висок throughput спрямо speculative decoding и предишни diffusion варианти, като за първи път затваря качествената дупка до AR при съществено по-висока скорост (порядък 4.71x-5.91x tokens/s по данни на NVIDIA).

3.4.0.1 Контекст: от класически speculative decode към TiDAR

Преди TiDAR най-честият practical подход за ускорение е класически speculative decoding с draft + verify логика. Този подход е добра отправна точка, но често страда или от по-слаб draft модел, или от по-ниска ефективност на верификацията.

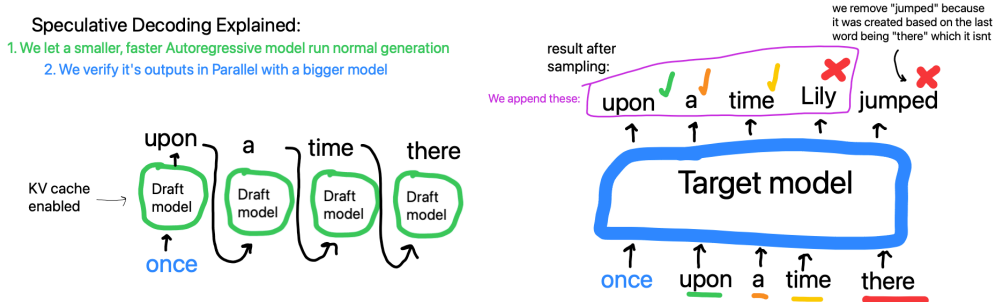


Figure 12: Базов speculative decoding (референтен случай)

В класическия вариант малък draft модел q и голям verify модел p работят с един и същ tokenizer и една и съща токена азбука. Draft моделът предлага последователност от кандидати $x_1, \dots, x_K$, а verify моделът ги проверява каузално за същия префикс.

$$\alpha_i = \min \left(1, \frac{p_i(x_i)}{q_i(x_i)} \right)$$

Тук α_i е вероятността за приемане на токена x_i на позиция i ; ако токенът се отхвърли, се семплира от коригираща дистрибуция:

$$r_{i(v)} = \frac{\max(0, p_{i(v)} - q_{i(v)})}{Z_i}$$

За KV cache е важно speculative токените да не се потвърждават окончателно предварително. Записва се само приетият префикс (или се прави rollback до приетата дължина), иначе cache състоянието се разминава с валидирания контекст и decode-ът деградира.

3.4.0.2 Основна идея на TiDAR в един forward pass

TiDAR променя тази схема, като комбинира verify + predraft в един structured forward pass. Така се използва по-добре наличният паралелен compute и се намалява serving overhead-ът.

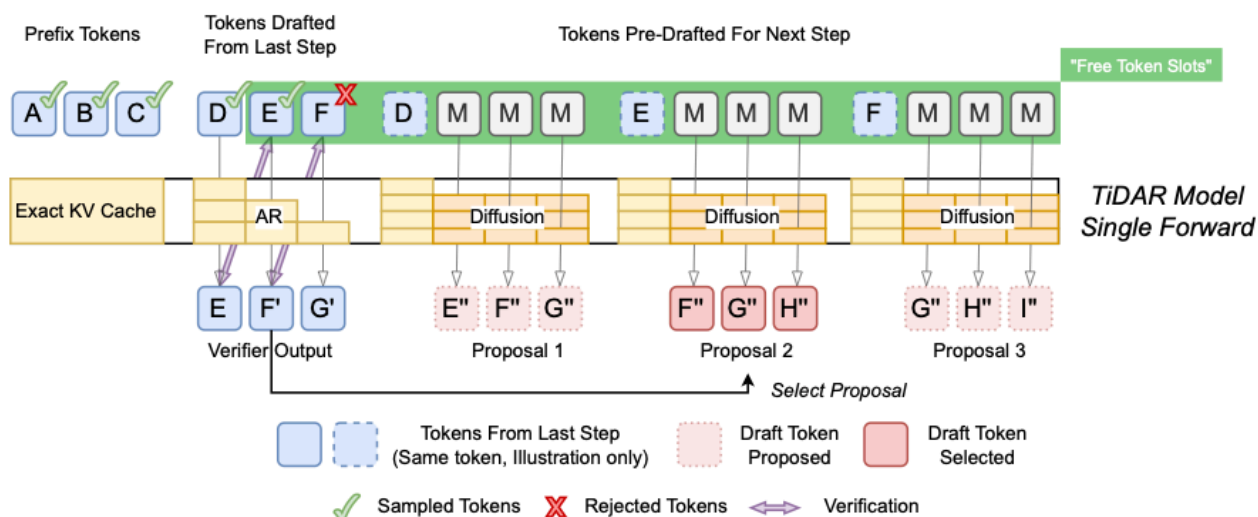


Figure 13: TiDAR single-forward layout: verify + predraft (diagram from TiDAR paper)

На схемата verify частта оценява текущия draft, а predraft частта едновременно предлага следващите кандидати за идната итерация. Ключовият момент е, че това не са два отделни модела, а един и същ TiDAR модел, изпълнен веднъж с различно структурирани attention връзки в рамките на същия forward pass.

Така се спестява допълнителен model orchestration overhead (няма втори draft модел и допълнителен cross-model sync), а наличният GPU compute се използва по-плътнo в една обща граф операция.

3.4.0.3 Структурирани маски и достъп по позиции

Ключът е в маските: различни части от входа имат различен режим на внимание, така че едновременно да се пази AR логиката за “talk” и diffusion паралелизъмът за “think”.

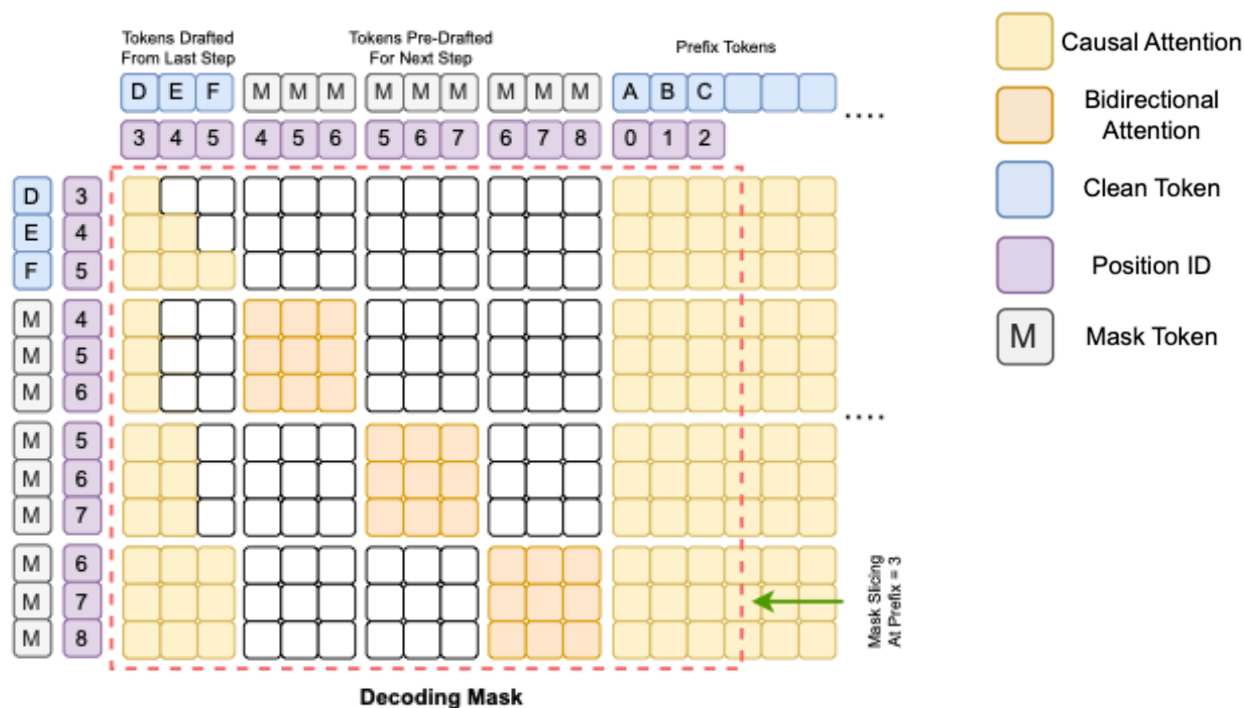


Figure 14: Структурирана маска за TiDAR decode/преддрафт логика (diagram from TiDAR paper)

Verify tokenите виждат каузален контекст по същия принцип както при нормален speculative decoding. Predraft tokenите виждат останалите токени в своя predraft group с bidirectional attention и едновременно с това виждат префикса до позицията, след която трябва да предсказват, т.е. до съответния draft token.

3.4.0.4 Prefill маска

Prefill фазата подготвя началното KV-cache състояние за decode, като подава prompt контекста с коректна каузална видимост. Това гарантира, че последващите verify/predraft стъпки стъпват върху консистентна базова история.

От prefill изхода се взима и първият D* token (първият draft token), който служи като стартова точка за следващата decode итерация.

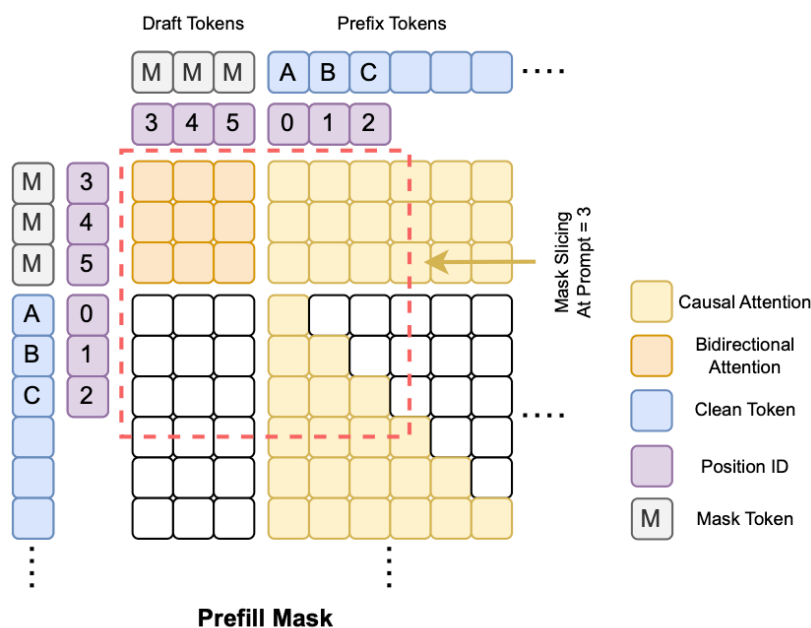


Figure 15: TiDAR prefill маска (diagram from TiDAR paper)

3.4.0.5 Training маска

В training режим маската реализира едновременно AR и diffusion режим върху подравнени позиции, така че моделът да учи и следващ token (causal), и паралелна denoising логика в diffusion частта.

Критичното предимство е, че training входът е с удвоен контекст [clean | diff] (приблизително 2S), а не с decode-подобна експлозия от типа $K + K^2$. Така TiDAR може да се тренира с по-управляем memory/compute профил и със стандартен training pipeline за фиксирана контекстна дължина.

В paper-а е тествано corruption поведение с шум, маска и смесени режими; отчетено е, че вариантът само с mask corruption работи най-добре. Затова и в тази имплементация се използва единен [MASK] token при diffusion частта на training/inference подредбата.

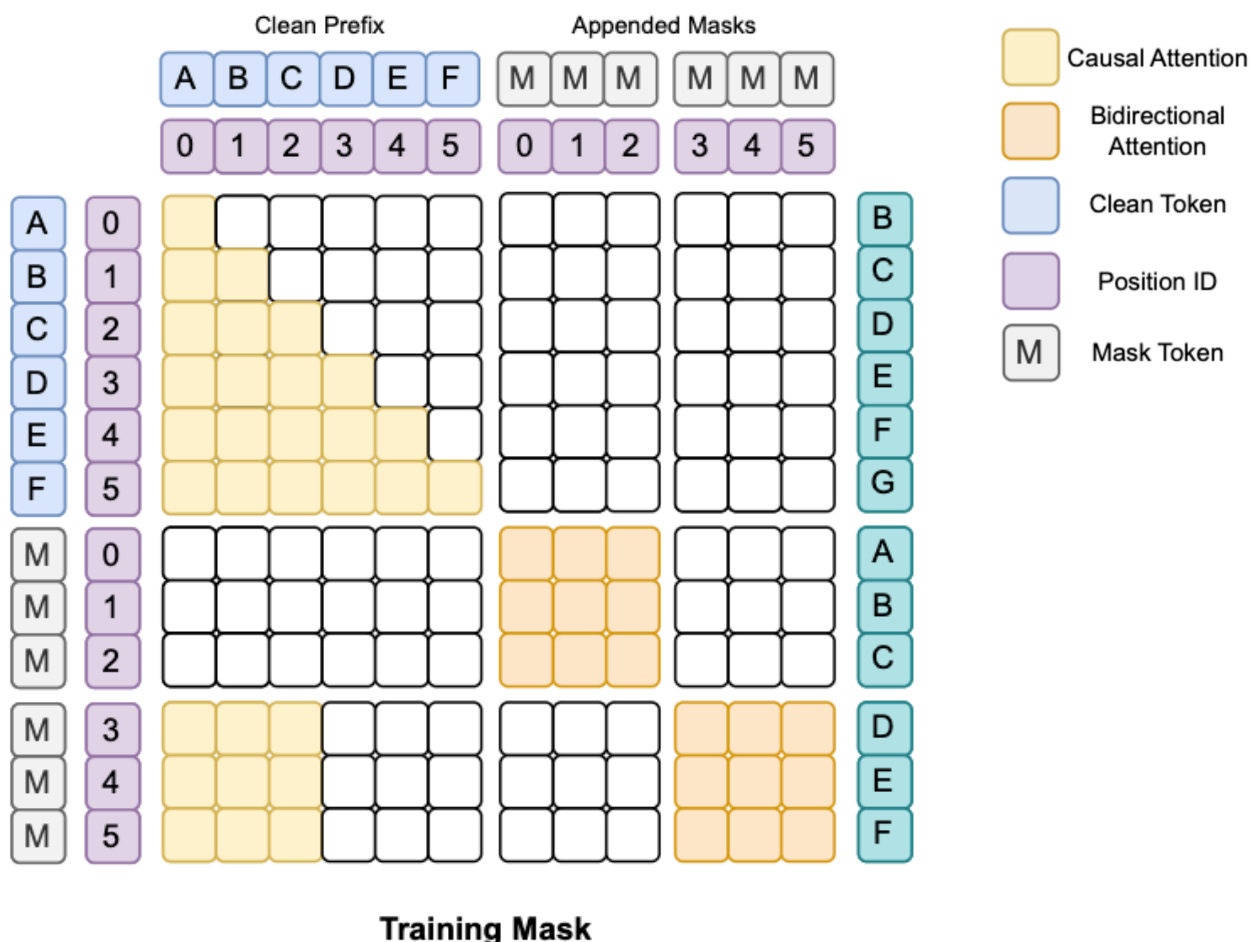


Figure 16: TiDAR training маска (diagram from TiDAR paper)

3.4.0.6 Loss формулировка и баланс AR:Diff

В оригиналната формулировка training целта е:

$$L_{T(\omega)} = \frac{1}{1 + \varepsilon} \left(\frac{\varepsilon}{S} \sum_{i=1}^S L_1(x_i, x_{i+1}; \omega) + \frac{1}{S} \sum_{i=1}^S L_2(m, x_i; \omega) \right)$$

където L_1 е AR терминът, L_2 е diffusion терминът, m е [MASK] токенът, а ε контролира баланса между двата терма.

В paper-а е направен sweep за съотношението между двата термина и е наблюдавано, че баланс 1:1 (т.е. $\alpha = \beta = 1$) е най-стабилен и дава най-добър общ компромис. Тестваният диапазон е около 0.8 .. 1.2 за всеки коефициент.

В тази работа се приема същият принцип: AR и Diff компонентите се държат балансирани като базова настройка, а отклоненията се използват само при целеви експерименти.

3.4.0.7 Anchor-TiDAR (финален акцент)

Anchor разширението въвежда стабилна референтна точка в decode стъпката и подобрява практическата ефективност на приемане/rollback логиката, особено при дълги генерации.

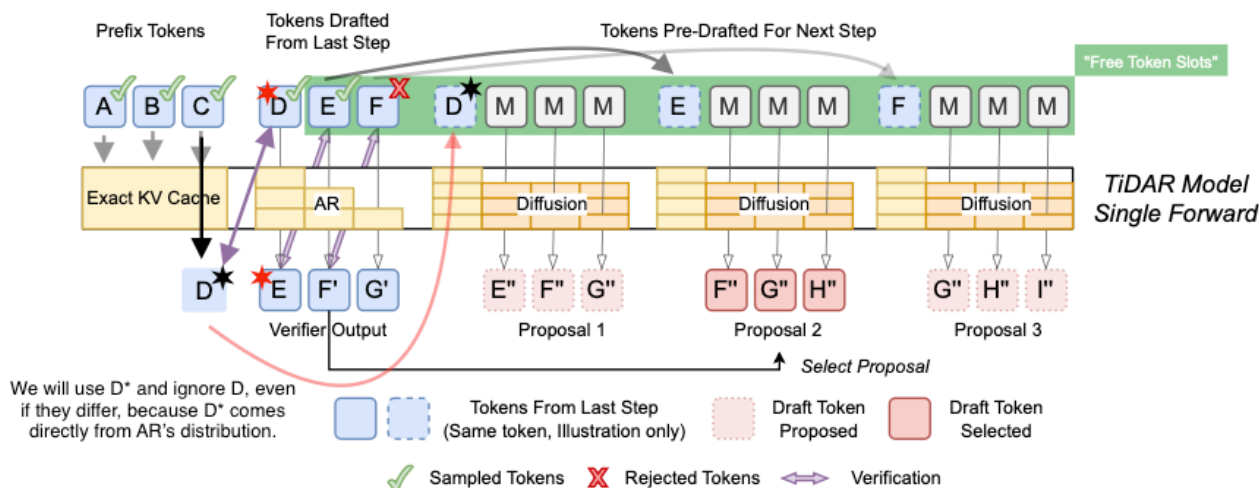


Figure 17: Anchor-TiDAR forward pass

Важно уточнение: Anchor-TiDAR е мое собствено разширение спрямо оригиналния TiDAR папер. В статията акцентът е по-скоро върху това моделът да е обучен достатъчно добре, така че пълен rejection (нула приети draft токени) да се случва минимално рядко, вместо да се описва изрично fallback механизъм за този случай. Ако все пак се случи full rejection, трябва или да се направи нов predraft pass, или да се държи допълнителен K набор (още compute), за да се избегне стоп в прогреса.

С Anchor подобрението се гарантира прогрес без нужда от допълнителен predraft compute в тази гранична ситуация, като целта е да се запази качеството на AR изхода. По-нататък е обяснено и как се управлява KV-cache pool-ът със static shape политика.

3.4.1 Представяне на dual sequence вход

TiDAR training конструира вход с дължина $2S$ във формат `[clean | diff]`:

- `clean` част ($1..S$) — стандартен AR поток за next-token предсказване;
- `diff` част ($S+1..2S$) — diffusion поток с [MASK] corruption и structured visibility.

На практика се учат две съгласувани задачи върху едни и същи таргети:

- AR: $x_i \rightarrow x_{(i+1)}$ по каузален ред;
- Diff: $\text{masked}(x_i) \rightarrow x_i$ в паралелен blockwise режим.

Това подравняване позволява директно сравнение и комбиниране на AR/Diff логити по позиции, което после се използва и при verify/predraft decode логиката.

Визуална референция за training маската: Figure 16.

3.4.2 Преобразуване и маскиране по позиции

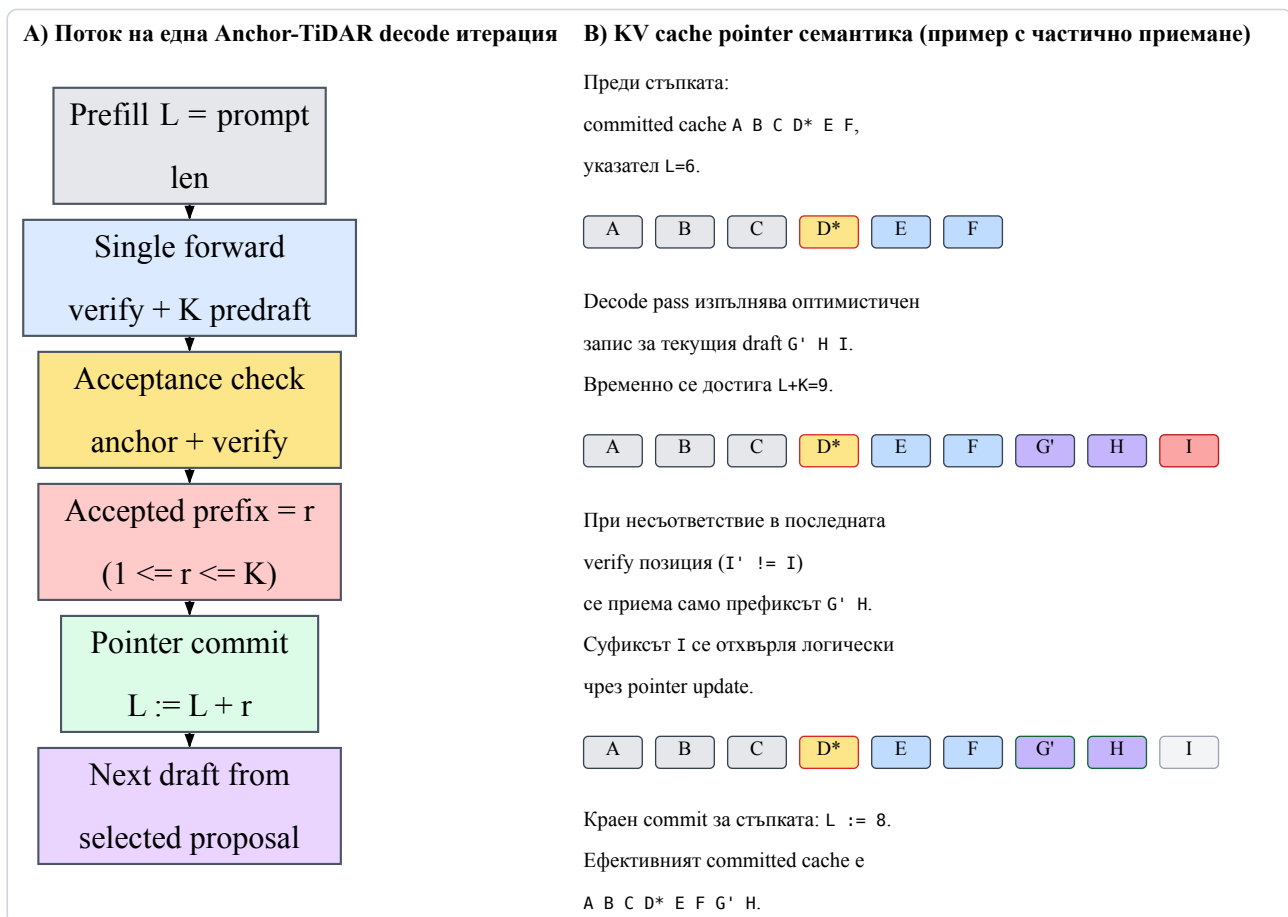
`tidar_masks.build_tidar_train_bias` създава attention bias матрица, която управлява точно кои позиции се виждат в training pass-a. Ключовите правила са:

- clean -> clean: каузален достъп (класически AR);
- diff вътре в block: bidirectional достъп;
- diff -> clean prefix: разрешен достъп до нужния контекст;
- clean/diff към бъдещи невалидни позиции: забранен достъп.

Тези правила гарантират, че AR частта остава каузално коректна, а diffusion частта получава паралелна локална видимост без leakage към бъдеща информация извън допустимия контекст. Decode/предрафт маската е показана по-горе на Figure 14.

3.4.3 KV-cache pointer commit/rollback

Anchor-TiDAR използва оптимистичен запис в KV cache и семантика с указател: приетият префикс се потвърждава чрез `prefix_len`, а отхвърленият суфикс се отстранява логически чрез връщане на указателя.



Listing 3: KV cache commit/rollback при Anchor-TiDAR: поток и pointer update

3.4.4 Извличане на verify и predraft логити

При inference входът се подрежда като `[current_draft | predraft_masks]`, след което от един forward pass се извличат:

- verify логити за текущия draft;
- K predraft предложения за следващата стъпка.

Технически това е важно, защото verify и predraft не се изпълняват като две отделни model извиквания. Логитите се взимат от различни позиционни срезове на един и същ output тензор, което:

- намалява host orchestration overhead;
- подобрява ефективността при static-shape изпълнение;
- поддържа консистентен KV-cache update модел за следващата итерация.

3.4.5 Decode сценарий (Anchor-TiDAR)

1. Prefill: записва се prompt в KV cache и се получава стартовият D^* .
2. Forward pass: едновременно се извличат verify логити и K predraft предложения.
3. Acceptance: предложенията се проверяват по ред и се приема най-дългият валиден префикс.
4. Commit/Rollback: KV pointer се премества само до приетата дължина.
5. Next step: от приетия префикс и новия anchor започва следващата итерация.

Визуални референции: Figure 17 и Figure 13.

3.4.6 Допълнителна визуална интерпретация за M_{ij}

Обозначаваме с A anchor токена, с $D1..Dk$ - draft токени, а с M_{ij} - токен в predraft блок i на позиция j .

M_{ij} вижда:

- каузално: префикса + anchor + нужния verify префикс;
- двупосочно: токените в собствения predraft блок;
- не вижда: нерелевантните бъдещи блокове извън текущата structured група.

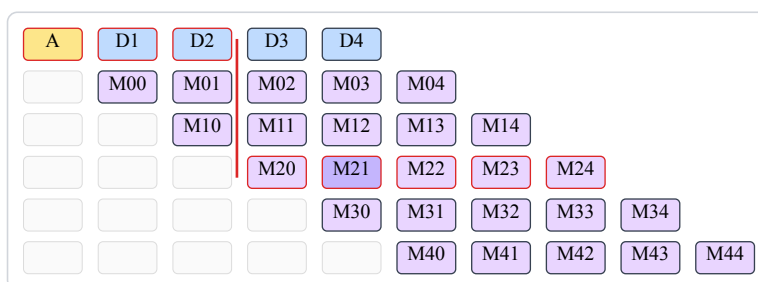


Figure 18: Триъгълна визуализация на verify/predraft зависимостите; пример с маркиран M_{21}

При токен M_{21} :

- каузално внимание: $prefix + A + D1 + D2$;
- двупосочно внимание: $M_{20}..M_{24}$ (в рамките на собствения блок);
- липса на внимание към останалите предрафт блокове.

Сравнение с класически speculative decode: Figure 12.

3.5 Експеримент: Free Token Slots

В този раздел се анализира явлението “Free Token Slots” - случаи, в които TiDAR може да използва допълнителна паралелна работа в една decode стъпка по-ефективно от класически AR decode път. Експериментите са върху A40 профили и са свързани директно с избора на structured masks, single-forward verify/predraft и KV-cache стратегията, описани по-горе.

В TiDAR paper-а има сходен тип анализ, но тук той е повторен върху значително по-слаб хардуер (A40 вместо H100), като е разширен и с допълнителни тестове за sampling режими и greedy decoding поведение.

Точно това явление е и основната мотивация зад начина, по който TiDAR е конструиран: да запълва “свободните” изчислителни слотове в decode стъпката с полезна verify/predraft работа, вместо GPU ресурсът да остава неизползван между последователните AR операции.

Важно за интерпретацията: показаните throughput резултати са benchmark-нати при зададен теоретичен accept_rate = 0.8 (около 80% приети токени на итерация). За тези експериментални матрици не са тренирани отделни нови модели за всеки сценарий; сравняват се скоростни профили при фиксирана теоретична асигура настройка.

3.5.1 Native decode attention

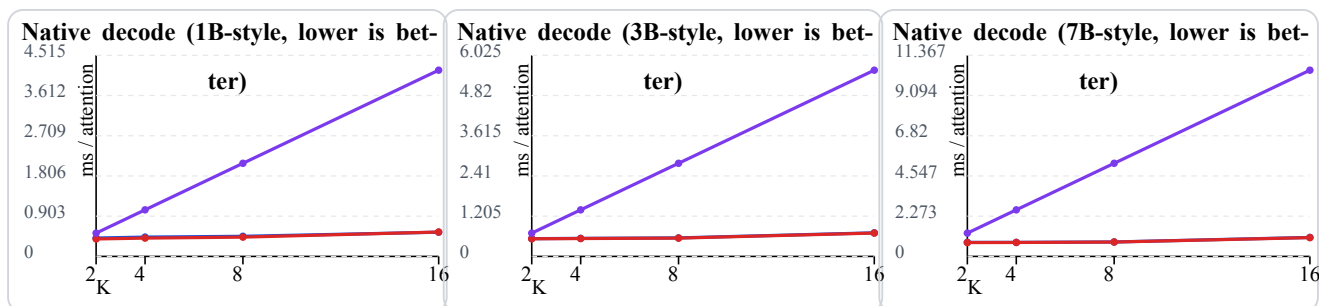


Figure 19: Free Token Slots - native decode attention при 1B, 3B, 7B (A40)

Легенда (decode attention графики):

- синя линия — TiDAR native structured (стандартният decode път със structured mask; почти припокрива се с червената);
- червена линия — TiDAR native dense / no-mask variant (същият decode път, но без подаване на structured mask; долната от двете близки TiDAR линии);
- лилава линия — AR baseline ($K \times \text{len1}$, горната линия).

Наблюдението тук е, че TiDAR native decode кривите остават значително по-плоски спрямо AR baseline, който е мащабиран като $K \times \text{len1}$. Това е practically важният сигнал за проекта: при нарастване на K се получава по-добра амортизация на compute разхода на стъпка и по-добро използване на наличния паралелен ресурс.

3.5.2 Kernel terms (контролен анализ)

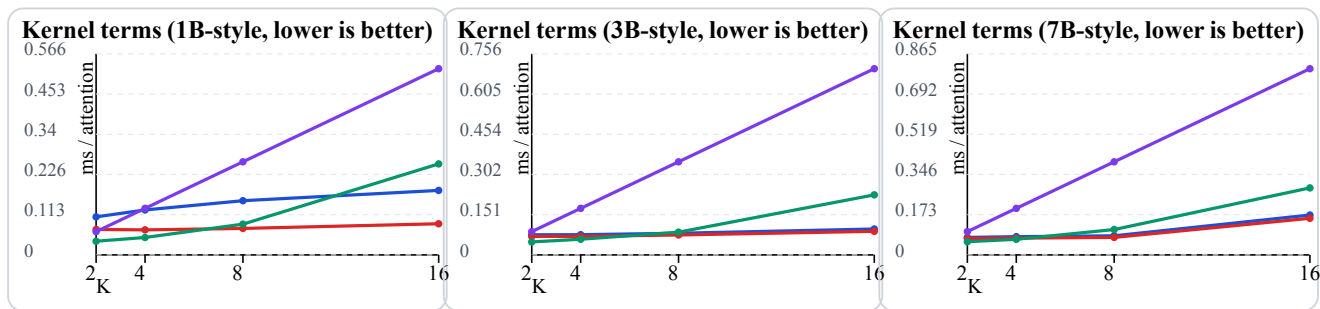


Figure 20: Kernel terms експеримент за attention pass варианти (1B, 3B, 7B)

Легенда (kernel terms графики):

- синя линия — kernel structured (реално използваният вариант);
- червена линия — kernel dense / no-mask (почти винаги най-долна);
- зелена линия — dense + jax-zero variant (при K=2,4 е най-долно, при K>4 започва да расте);
- лилава линия — AR baseline ($K \times \text{len1}$, консистентно най-горна права линия).

Тези графики са контролен експеримент за различни attention pass варианти и целят да изолират ефекта на маската върху compute профила. Важно: тук не се влиза в custom kernel посока (Pallas/Triton/собствени CUDA/C++ kernel-и); анализът е на текущия production-like decode път. Както и в paper-a, custom kernel оптимизациите остават по-скоро посока за бъдещо развитие.

3.5.3 MLP scaling

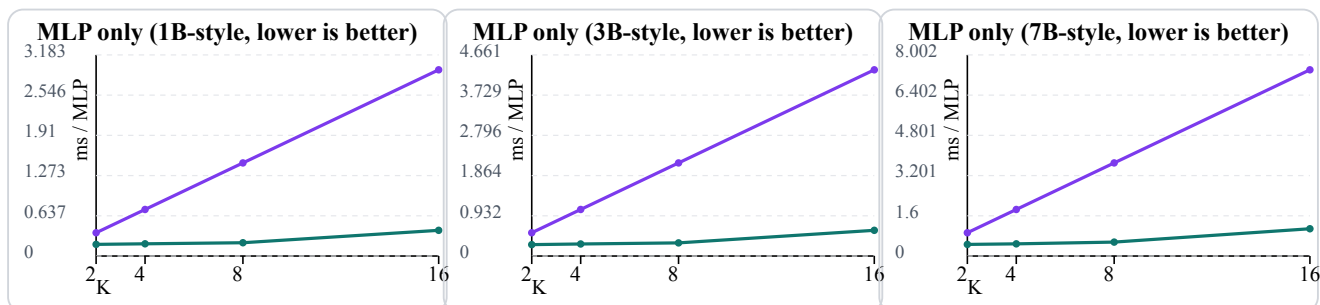


Figure 21: Free Token Slots - MLP scaling сравнение (1B, 3B, 7B)

Легенда (MLP графики):

- тюркоазена линия — TiDAR MLP при $L = K + K^2$ (долната линия);
- лилава линия — AR MLP baseline ($K \times \text{len1}$, горната линия).

MLP графиките подсилват същата идея: при TiDAR токеният блок $L = K + K^2$ се обработва в един общ pass, докато AR baseline се акумулира почти линейно с K. За текущата архитектура това е ключова връзка между теорията и практиката - structured single-model decode пътят не само работи коректно, а и носи реална throughput полза в настройките, върху които е разработен проектът.

3.5.4 Sampling path (greedy и non-greedy)

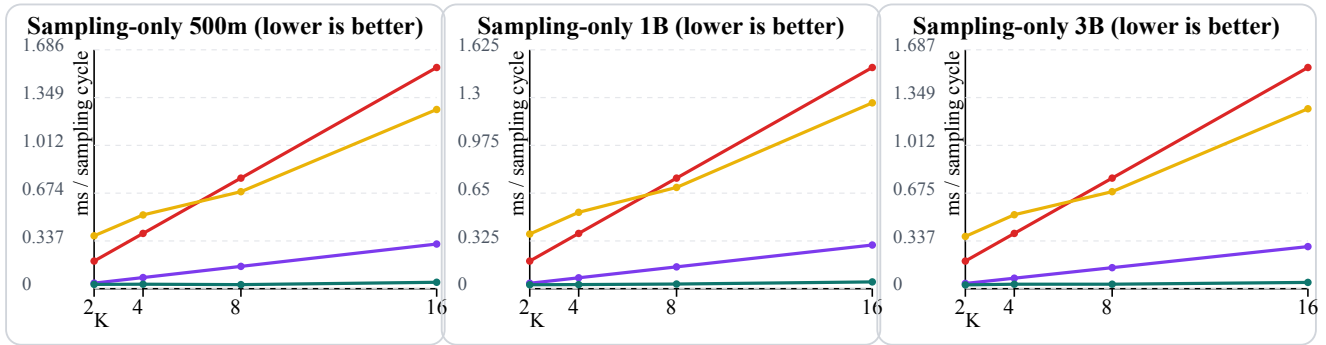


Figure 22: Free Token Slots - sampling-only сравнение (500m, 1B, 3B)

Легенда (sampling графики):

- тюркоазена линия — TiDAR staged sampling, greedy ($\text{top}_k=0$);
- лилава линия — AR scaled baseline, greedy ($K \times \text{len1}$);
- жълта линия — TiDAR staged sampling, non-greedy ($\text{top}_k=50$);
- червена линия — AR scaled baseline, non-greedy ($\text{top}_k=50$).

Sampling-only резултатите показват, че при greedy режим TiDAR държи по-нисък sampling cycle почти по целия диапазон на K . При non-greedy ($\text{top}_k=50$) има crossover поведение: при ниски K разликата е малка/в полза на AR, а при по-високи K TiDAR започва да печели, което е в синхрон с целта да се използва по-добре паралелният compute при по-широк draft.

Как работи staged sampling в този benchmark: първо се семплира verify токенът за текущата позиция, след което се прави rejection/select стъпка за predraft предложенията и се семплира само избраният predraft ред за следващата итерация. Така се мери изолирано именно sampling/rejection пътят (без transformer forward), което позволява директно сравнение с AR $K \times \text{len1}$ baseline.

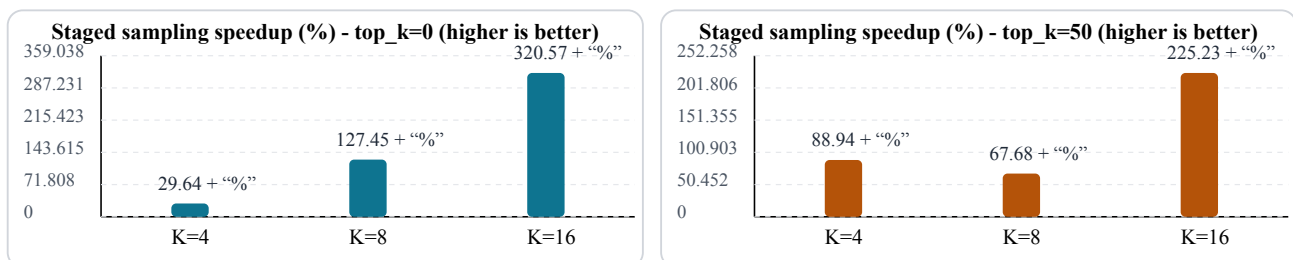


Figure 23: Изолиран sampling speedup: staged path спрямо old path

Практически извод: в тези run-ове sampling частта е малък дял от целия decode wall-time (приблизително под 1%). Това се вижда от факта, че изолираният sampling speedup е голям, но end-to-end decode подобрението остава около десети от процента; следователно основният bottleneck остава transformer forward пътят, а не самото sampling действие. Въпреки това free

token slots остават важни, защото показват къде има неизползван паралелен ресурс и къде следващите оптимизации могат да донесат допълнителен throughput.

3.6 Резултати: TiDAR срещу AR (A40 и H100)

След анализа на Free Token Slots тук показваме head-to-head резултатите за TiDAR срещу AR при еднаква 3b конфигурация върху A40 и H100.

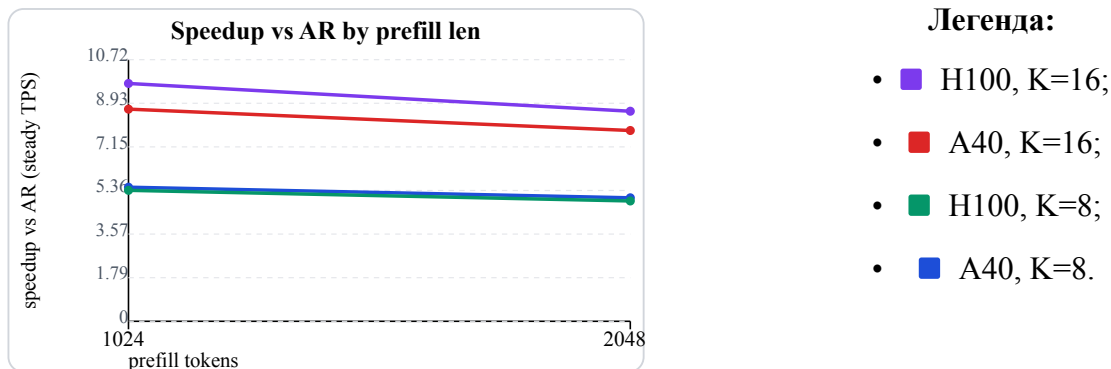


Figure 24: Head-to-head: speedup vs AR по prefill (A40 + H100)

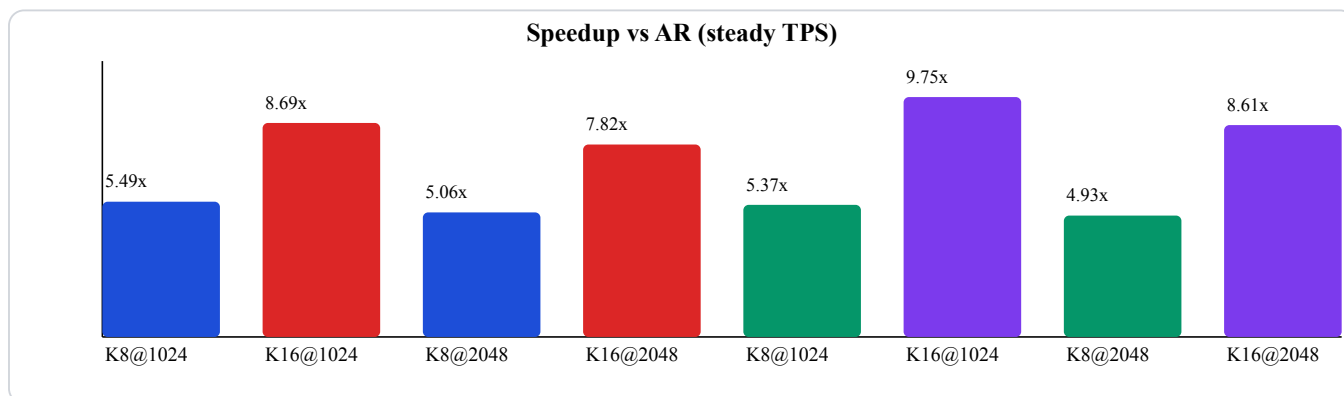


Figure 25: Head-to-head: обобщена speedup диаграма (full width)

Тези резултати потвърждават, че наблюдаваните ускорения не са единичен артефакт от една карта, а се запазват и при по-висок клас GPU, като при K=16 H100 показва най-силна speedup крива.

3.7 Разширена loss функция за TiDAR

Тази секция описва разширената loss формулировка в имплементацията, при която отделните обучителни термини са с независими коефициенти и могат да се комбинират контролирано според целта на конкретния training stage.

3.7.1 Формулировка с независими коефициенти

В практическата имплементация замених нормализацията от $\text{ramp-a } \frac{1}{1+\alpha}$ с модулна сума от отделни термини, всеки със собствен коефициент. Така всеки компонент може да се усилюва, отслабва или да се изключва с нулев коефициент, което прави възможни чисти ablation експерименти и целенасочени комбинации между различни обучителни сигнали.

Легенда за сравняваните модели:

- ■ 135M модел - run на RTX 5090 (32GB), платена цена за наем 5.23 USD;
- ■ 360M модел - run на RTX PRO 6000 (96GB), платена цена за наем 11.81 USD.
- ■ сива линия = делта 135M / 360M (1.0 означава равни стойности).

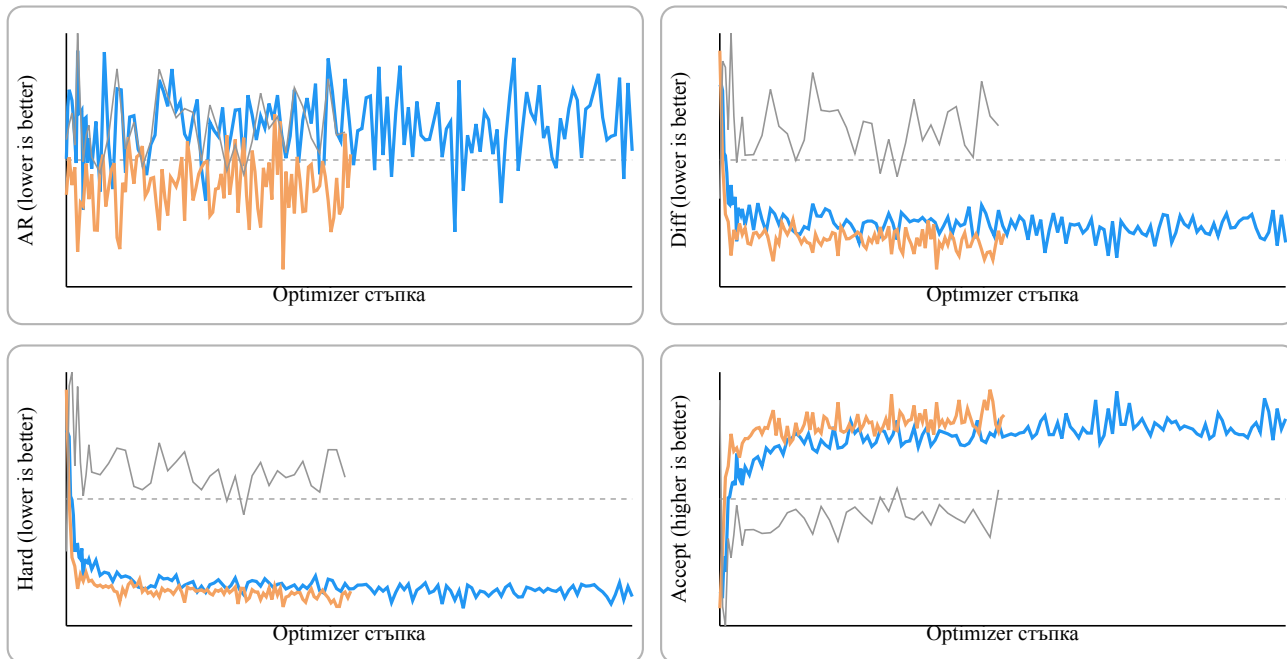


Figure 26: Обучителни метрики: 135M срещу 360M

Практически извод: 360M вариантът държи по-стабилни и по-ниски Diff/Hard загуби и по-висок greedy acceptance, но идва с осезаемо по-висока цена във време и ресурс. Моделът е приблизително 2.66 пъти по-голям по брой параметри спрямо 135M, а в реалните run-ове обучението му беше около 3-4 пъти по-бавно. Тези диаграми показват и че началните способности на базовия модел директно се отразяват върху ученето на TiDAR future prediction-ите: по-силният стартов модел учи по-стабилно тази задача, което е консистентно с по-големия му капацитет и с факта, че е предварително трениран върху повече данни. За проекта това е важен ориентир за trade-off между качество, цена и време за итерация при TiDAR training.

3.9 Допълнителен експеримент: TinyStories attention head sweep

Проведен е кратък head sweep върху TinyStories с четири конфигурации: h6 MHA (baseline), h18 MHA, h18/kv6 GQA и контролен вариант h6 с head_dim=32. Целта е бърза проверка дали промяната в attention head конфигурацията води до съществено разделяне по GIANT/TiDAR обучителните метрики.

Легенда на конфигурациите:

- h6 MHA (baseline)
- h18/kv6 GQA
- h18 MHA
- h6 head_dim=32

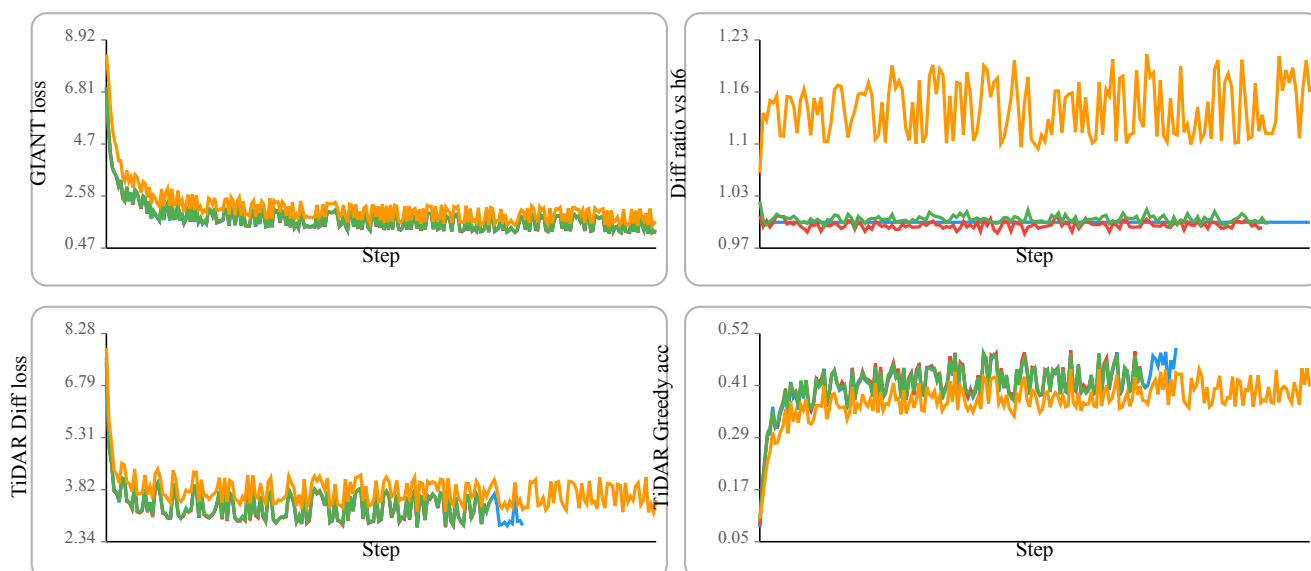


Figure 27: TinyStories head sweep: компактен преглед на 4 ключови обучителни метрики

Резултатите показват ограничено разделяне между сравнимите по ширина конфигурации (h6/h18/GQA) в рамките на краткия run прозорец. Увеличаването на броя heads само по себе си не дава ясно доминираща печалба по качество за този setup, докато GQA остава близо до baseline при умерено по-добра ефективност. Контролният h6 head_dim=32 режим е значително по-бърз, но с отчетлив спад в качеството, поради което се интерпретира като trade-off вариант, а не като пряк “по-добър” заместител.

3.10 Проведен експеримент с различни loss функции и конфигурации

В този раздел е представен контролиран експеримент за TiDAR post-training с множество loss конфигурации. Основната цел е да се оцени кои комбинации подобряват ключовите метрики (Diffusion loss и Greedy acceptance), като едновременно с това се запази качеството на оригиналния AR модел (без значимо влошаване на AR loss спрямо последните итерации от базовото GIANT обучение). Дизайнът е избран така, че да поддържа бърз експериментален цикъл: промяна на loss конфигурация, кратък run, метрика-анализ и следваща итерация.

3.10.0 Избор на TinyStories и базова конфигурация

За експерименталната серия е избран TinyStories, защото е сравнително малък корпус с ниска ентропия. Това позволява бързи итерации и ясна интерпретация на ефекта от loss промените. В практиката този тип корпус се научава стабилно и от базов модел около 30M параметъра, без да е необходимо пълно минаване през целия набор във всеки run.

В конкретната серия базовият GIANT модел (30M параметъра) е обучен върху около 1.05 милиарда токена за приблизително 33 минути на RTX 5090 (0.89 USD/час), а всеки TiDAR post-training run е отнемал около 2.5 часа на същия хардуер. Общата експериментална кампания е

проведена в рамките на една седмица с междинен анализ след всеки run (включително логитни хистограми), при приблизителен общ разход около 32 USD.

Базовият GIANT модел в тази серия е дефиниран със следната конфигурация:

```
model:
  embedding_size: 384
  num_heads: 6
  num_kv_heads: 2
  num_layers: 18
  feed_forward_size: 1024
  context_length: 512
training:
  batch_size: 32
  gradient_accumulation: 4
stages:
  - dataset: tinystories_512
    seq_len: 512
    epochs: 2
    fraction: 0.75
```

След базовото AR обучение се преминава към TiDAR post-training със същата архитектура, като началният режим е $\alpha=1$, $\beta=1$:

```
tidar:
  draft_length: 6
training:
  batch_size: 16
  gradient_accumulation: 4
loss:
  alpha: 1.0
  beta: 1.0
  rho: 0.0
  chi: 0.0
  delta: 0.0
  eta: 0.0
stages:
  - dataset: tinystories_512
```

3.10.2 Сравнявани loss режими

Легендата по-долу обобщава сравняваните конфигурации в sweep-а. Тя покрива както “меки” alignment сигнали (KL/Distill), така и “твърди” acceptance-ориентирани сигнали (delta, delta_masked, top-k set), което е методологично правилно за TiDAR, където целта не е само нисък loss, а по-ефективен speculative decode.

Легенда на вариантите:

-  Stable - $\alpha=1$, $\beta=1$, референтна линия;
-  KL-only - $\alpha=0.2$, $\beta=0$, $\rho=0.5$, $\delta=0.3$, без Diff CE;
-  KL-keep - $\alpha=1$, $\beta=1$, $\rho=0.1$, $\delta=0.3$, умерен alignment;
-  Distill - $\alpha=1$, $\beta=1$, $\eta=0.04$, $T=2$, мека дистилация AR->Diff;
-  Greedy eta - агресивен agreement/дистилация режим;
-  Bigger beta - $\alpha=1$, $\beta=5$, усилен Diff натиск;
-  Top-K set - $\gamma=0.01$, $\gamma_{\text{topk}}=8$, късен stage;
-  Big Gamma+Delta - силен Top-K + hard agreement;
-  Masked Delta later - частичен 90k+ run с delta_masked;
-  Delta masked early - кратък ранен run под 30k.

Run-ове, които водят до твърде бърза деградация на AR quality (catastrophic forgetting) или показват незначим ефект спрямо основната група, не са включени във финалното сравнение в тази секция.

3.10.3 Интерпретация на AR и Diff кривите

AR панелите показват, че стабилният режим и умерените добавки (KL-keep, Distill) запазват близка и устойчива AR динамика в 90k-121k. Това означава, че тези варианти не разрушават основната next-token способност.

Diff панелите дават по-силен разделителен сигнал:

- при KL-only ($\beta=0$) Diff loss след 90k става нефункционален като сравним индикатор (затова е изключен от zoom Diff панела);
- KL-keep и Distill остават в сравним диапазон със stable, което е знак, че дифузионният клон продължава да учи полезно, докато се добавя alignment.

Изводът е, че пълното изключване на Diff CE може да повиши някои acceptance метрики краткосрочно, но отслабва контрола върху самата Diff реконструкция и прави режима по-рисков за дълги run-ове.

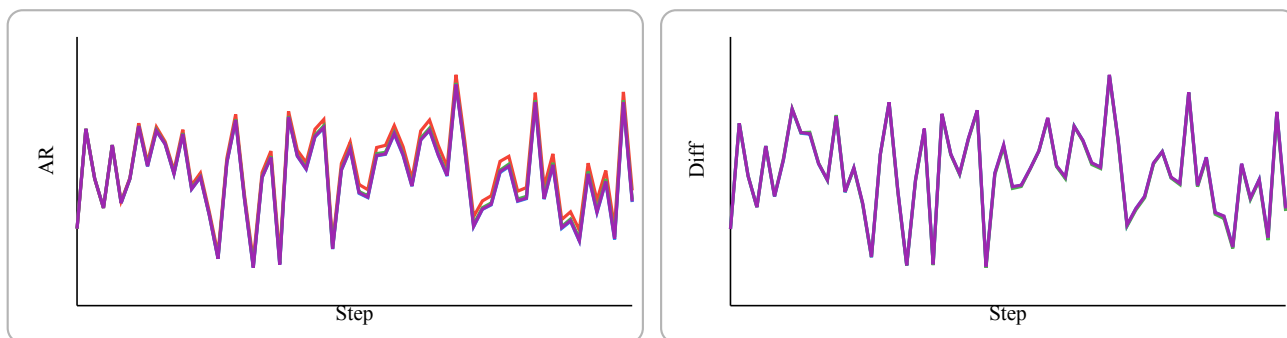


Figure 29: AR и Diff zoom сравнение за 90k-121k

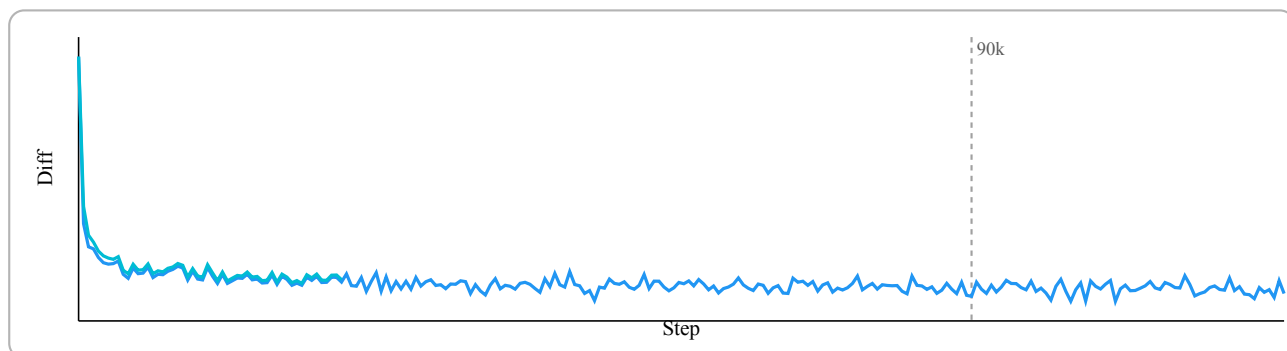


Figure 30: Пълен Diff прозорец: stable срещу ранен delta_masked run

3.10.4 Greedy acceptance: основен таргет на sweep-a

Greedy acceptance е централната метрика в документа ($\text{argmax}(\text{AR}) == \text{argmax}(\text{Diff})$ по валидни позиции). Показаните панели имат три нива на интерпретация:

- absolute панели (90k-121k): сравнение на реалните траектории;
- delta панели спрямо stable: видимост дали вариантът е устойчиво над/под baseline;
- extended панели: добавят агресивни режими и по-късни експериментални хипотези.

Най-важните наблюдения за практиката са:

- KL-keep и Distill дават по-добър баланс между acceptance печалба и запазена training стабилност;
- KL-only може да покаже моментни плюсове в greedy, но на цената на “изключен” Diff обучителен сигнал;
- по-агресивните режими (напр. Big Gamma+Delta) изискват внимателно stage планиране, защото увеличават чувствителността към хиперпараметри;
- delta_masked (късен и ранен вариант) е концептуално обещаващ, защото натиска точно acceptance-критичния префикс, но частичният характер на част от run-овете налага предпазлива интерпретация.

Част от run-овете са прекратявани по-рано, когато се наблюдава силно покачване на AR loss (catastrophic forgetting) или когато Diffusion/Greedy метриците тръгват устойчиво под основната

група. Поради това някои криви са по-къси и не достигат до последните точки от общия прозорец. Всички логове и checkpoint-и от тези run-ове са запазени в репото.

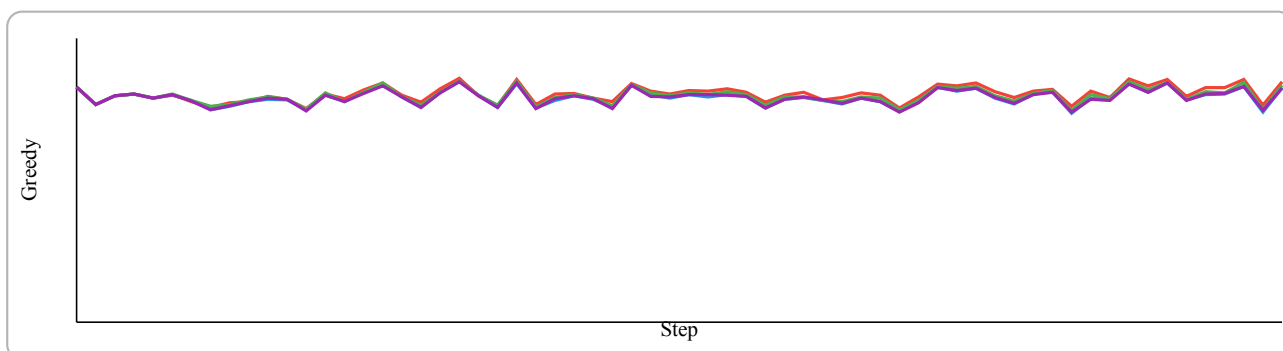


Figure 31: Greedy acceptance: core варианти в 90k-121k

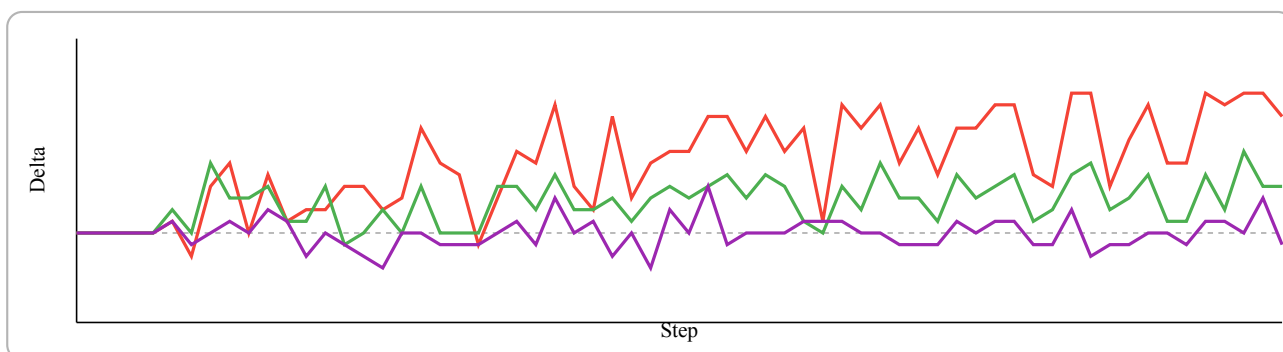


Figure 32: Delta срещу stable за core вариантите

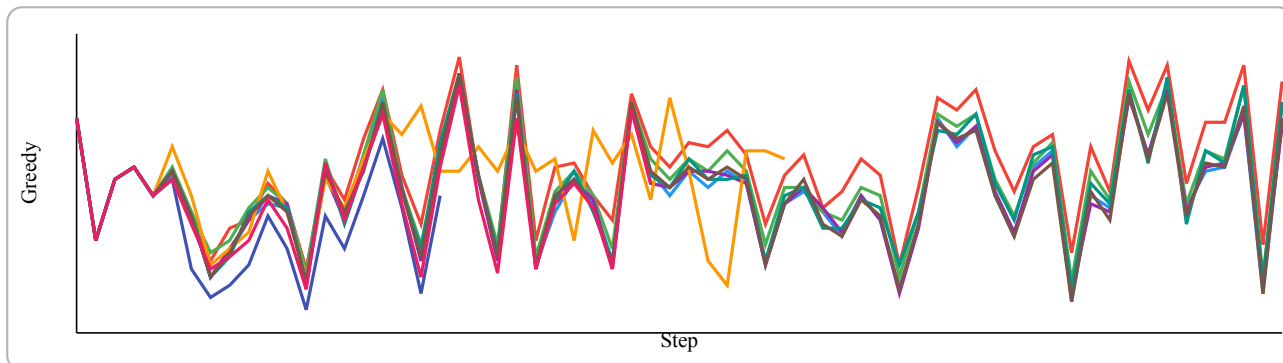


Figure 33: Greedy acceptance: разширен набор от всички варианти

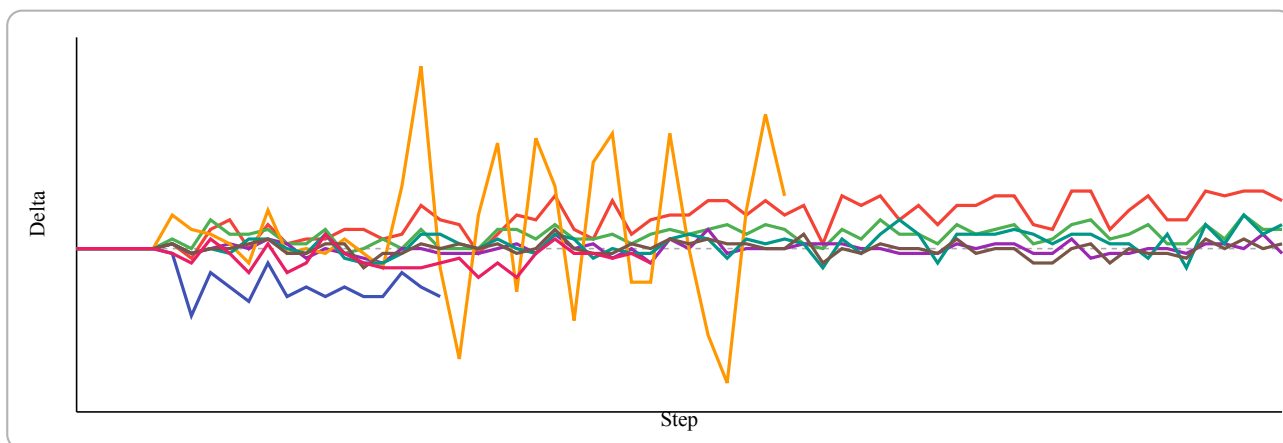


Figure 34: Delta срещу stable за разширения набор

3.10.5 Inference резултати (Accept/Iter таблица)

Секцията “Inference Results” проверява дали training ефектите се пренасят при реален greedy decode (temperature=0, draft_len=6) върху 10 промпта и три дължини на генериране (50/100/300 стъпки). Ключовите тенденции в таблицата са:

Checkpoint	Steps=50	Steps=100	Steps=300
Stable @90k	2.41 / 3.50 / 1.63	2.26 / 2.75 / 1.71	2.10 / 2.45 / 1.88
Stable @121k	2.22 / 3.27 / 1.53	2.09 / 2.68 / 1.60	2.10 / 2.69 / 1.57
KL-only @121k	2.42 / 3.27 / 1.81	2.30 / 2.68 / 1.80	2.08 / 2.49 / 1.61
KL-keep @121k	2.40 / 3.27 / 1.75	2.29 / 2.83 / 1.80	2.18 / 2.90 / 1.75
Distill @121k	2.31 / 3.50 / 1.53	2.26 / 3.09 / 1.87	2.19 / 2.74 / 1.80
SmallAR+eta @105k	2.15 / 2.88 / 1.75	2.15 / 2.75 / 1.90	2.00 / 2.43 / 1.74
Bigger beta @121.5k	2.37 / 3.27 / 1.63	2.33 / 3.09 / 1.62	1.96 / 2.27 / 1.74
Top-K set @121k	2.28 / 3.12 / 1.52	2.18 / 2.56 / 1.72	2.19 / 2.91 / 1.56
Big Gamma+Delta @105k	2.32 / 3.33 / 1.67	2.26 / 2.86 / 1.79	2.09 / 2.59 / 1.71
Masked Delta later @105k	2.30 / 3.12 / 1.72	2.34 / 2.86 / 1.72	2.01 / 2.40 / 1.67

Table 1: Accept/Iter summary върху 10 TinyStories промпта

- при Steps=300 най-силен среден резултат дават Distill (2.19) и Top-K set (2.19), следвани от KL-keep (2.18), над Stable (2.10);
- при Steps=100 Stable bigger beta достига висока средна стойност (2.33), но при дългия хоризонт (300) пада до 1.96, което подсказва по-слаба устойчивост;
- SmallAR big greedy eta и Masked Delta later са по-агресивни режими и остават под най-добрите устойчиви варианти при по-дълги генерации.

Това потвърждава работната хипотеза, че умереният alignment (KL-keep или Distill) е по-надежден за обща decode ефективност от прекалено силни наказания/насърчения върху отделен компонент.

3.10.6 Локален механистичен анализ чрез logits хистограми

Логитните хистограми дават позиционен анализ на механизма за accept/reject и допълват агрегираните метрики с локална причинна интерпретация. Всяка група панели представя една decode итерация за конкретен промпт и конкретен позиционен диапазон от draft блока.

За всяка позиция се показват два панела: горният е AR verify разпределението, а долният е Diff draft разпределението за същите токени в същия ред. По този начин сравнението е директно по позиции и по кандидати, без повторно сортиране в долния панел. Практически целта е да се види къде AR и Diff остават съгласувани и в кой момент започва разминаване, което прекъсва acceptance веригата.

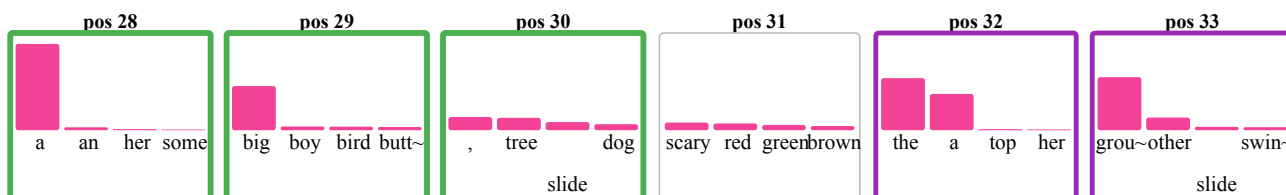
Легенда на визуализацията:

- розови стълбове: AR verify разпределение;
- сини стълбове: Diff draft разпределение;
- зелена рамка: позицията е приета в acceptance веригата;
- лилава рамка: AR top-1 съвпада с Diff top-1, но позицията не е приета;
- сива рамка: липсва top-1 съвпадение.
- в ранните позиции на draft блока AR и Diff често съвпадат по top-1 и токените се приемат;
- при нееднозначни позиции Diff измества top-1 и започват rejection-и;
- точно тези позиции обясняват защо prefix-ориентирани loss-и (като `delta_masked`) имат смисъл: те таргетират момента, в който acceptance веригата се прекъсва.

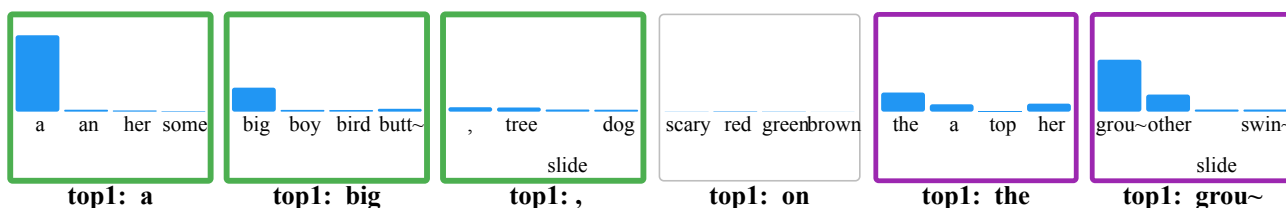
В показаните примери анализът е фокусиран върху по-късен контекст (приблизително след позиция 28), тъй като в началните позиции acceptance обикновено е близък до 100% и наблюдението е по-слабо информативно.

Prompt: “Once upon”

AR verify logits (top-4, sorted by AR)



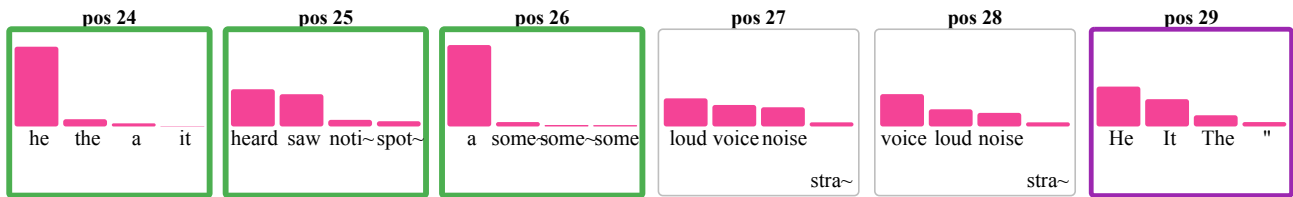
Diff draft logits (same tokens, AR order)



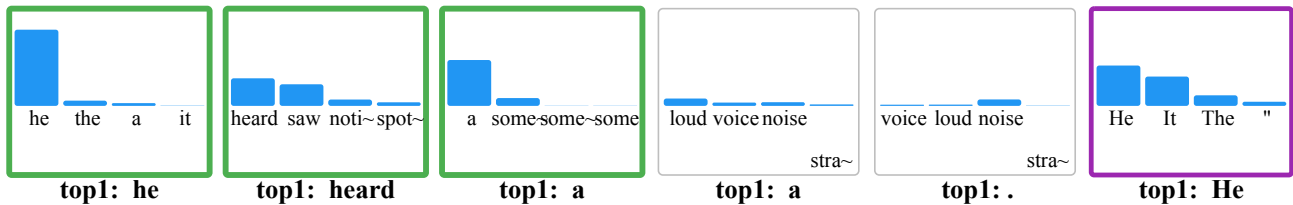
Sentence context One day, she saw a big, scary dog.

Prompt: “In the forest, a tiny fox”

AR verify logits (top-4, sorted by AR)



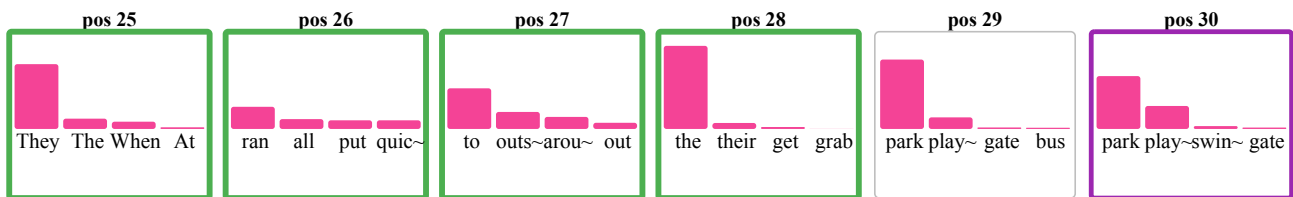
Diff draft logits (same tokens, AR order)



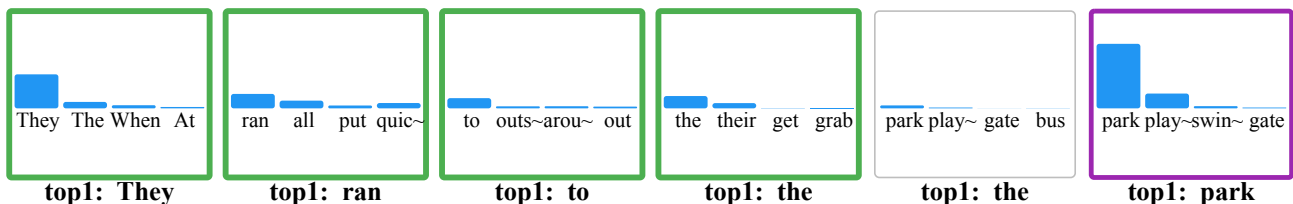
Sentence context Suddenly, he heard a loud noise.

Prompt: “One day, the teacher said”

AR verify logits (top-4, sorted by AR)



Diff draft logits (same tokens, AR order)



Sentence context They ran to the park and saw a big slide.

Наблюдаваното натрупване на лилави рамки след първото несъответствие е очаквано за TinyStories, защото корпусът е силно структуриран и с ограничена лексикална вариативност. След прекъсване на acceptance веригата (първи reject) локалният контекст често остава достатъчно ясен, така че AR и Diff отново дават един и същ top-1 кандидат в следващи позиции. Това локално съвпадение обаче не възстановява автоматично вече прекъснатия префикс за текущата итерация, поради което маркировката остава лилава, а не зелена.